Thesis Plan

Decentralized Multi-Agent Reinforcement Learning for Power Grid Topology Control

Corentin Plumet

August 4, 2026

Contents

Chapter 1

Thesis Plan

1.1 Plan Inspiration

This plan follows the structure of the reference thesis in `mAI2024deMolB.pdf`: start from the power-grid motivation and research questions, introduce the scientific background, describe the environment before the algorithms, then separate methods from results and close by answering the research questions. The adapted structure below is tuned to this repository: Grid2Op topology control, decentralized MAPPO, graph encoders, rerun ablations, and sparse intervention mechanisms.

1.2 Working Title

Decentralized Multi-Agent Reinforcement Learning for Sparse Power Grid Topology Control

1.3 Central Thesis Question

How can decentralized multi-agent reinforcement learning agents learn reliable power-grid topology control policies that preserve survival performance while scaling to combinatorial action spaces and avoiding unnecessary interventions?

1.4 Research Questions

1. How should the Grid2Op topology-control problem be formulated as a decentralized multi-agent decision problem?
2. Which architectural choices improve survival and generalization: flat MLP policies, graph neural network encoders, shared actor weights, critic design, or optimized critic updates?
3. How do initialization and training choices, especially the initial do-nothing bias and reward normalization, affect exploration and learned behavior?
4. Can sparse-control mechanisms, such as rho-based overrides, intervention gates, fixed penalties, or adaptive intervention budgets, reduce unnecessary topology actions without sacrificing episodic survival?

1.5 Expected Contributions

- A clear decentralized formulation of topology optimization in Grid2Op.

- A controlled comparison of MAPPO baselines, GNN encoders, critic variants, and initialization choices.
- An ablation study of sparse intervention mechanisms for more operator-like control.
- A reproducible experiment and analysis pipeline based on W&B histories, cached metrics, notebooks, and final evaluation scripts.

1.6 Chapter Outline

Chapter 1: Introduction

Goal. Motivate topology control as a low-cost but difficult operational tool for power-grid reliability.
Planned content.

- Energy-transition context: higher demand, renewable uncertainty, and stressed transmission networks.
- Topology control as an alternative or complement to redispatching and grid expansion.
- Why the problem is hard: sequential control, physical constraints, rare failures, and combinatorial topology actions.
- Why decentralized MARL is a natural fit: local substations, regional decision making, reduced per-agent action spaces, and scalability.
- Research questions, contributions, and thesis outline.

Chapter 2: Background and Related Work

Goal. Give the reader enough power-systems and RL background to understand the design choices.
Planned content.

- Power-grid operation: substations, busbars, line loading, overloads, contingencies, and remedial actions.
- Grid2Op and the Learning to Run a Power Network setting.
- Reinforcement learning basics: MDPs, policy gradients, PPO, and MAPPO.
- Multi-agent reinforcement learning: decentralized execution, centralized training, credit assignment, and non-stationarity.
- Graph neural networks for grid-structured observations.
- Safe, sparse, and constrained RL ideas relevant to intervention budgets.

Chapter 3: Environment and Problem Formulation

Goal. Define exactly what the agents observe, choose, optimize, and are evaluated on.
Planned content.

- Grid2Op environment used in the repository and the selected chronics.
- Multi-agent decomposition: agent-region or agent-substation assignment.
- Observation spaces: local grid state, line loadings, topology state, and optional graph observations.
- Action spaces: local topology actions, action 0 as do nothing, legal action constraints, and action-space reduction.
- Reward and evaluation metrics: episodic survival, illegal action rate, action-0 fraction, non-idle fraction, and train/test chronic split.
- Reproducibility setup: seeds, W&B logging, cached histories, and deterministic evaluation.

Chapter 4: Methods

Goal. Present the implemented learning systems and the ablations.

Planned content.

- MAPPO baseline: actor, critic, rollout collection, PPO objective, and centralized training assumptions.
- Baseline MLP agents and the MLP baseline family of reruns.
- Graph-encoder agents: bus graph construction, GINE/GAT/GCN variants, node and edge pre-encoders, graph readout, and flat-feature concatenation.
- Action heads: the action-index MLP head and the candidate-action head that scores each action from the explicit-line graph nodes it touches.
- Critic variants: GNN critic versus MLP critic, legacy versus optimized critic updates.
- Initialization and exploration choices: entropy coefficient, entropy decay, and initial do-nothing probability.
- Sparse-control mechanisms: evaluation-time rho heuristic, intervention gate, fixed sparse-action penalties, and adaptive intervention budgets.
- Experimental protocol for ablations: controlled seeds, fixed evaluation cadence, and fair baseline comparisons.

Chapter 5: Experiments and Results

Goal. Show which design choices matter and what behaviors the agents learn.

Planned content.

- Main baseline performance: training stability and test survival.
- GNN architecture ablations: concat-flat, actor sharing, GINE versus GAT or weighted GCN, light versus full encoders, and critic design.
- Action-head ablation: candidate-action scoring versus the action-index head on the explicit-line graph, on `bus14` first and then on the reduced WCCI action space.
- MLP baseline rerun ablations: optimized critic updates, smaller MLP, reward normalization, and do-nothing initialization bias.
- Sparse-control ablations: fixed penalty, global-safe weighting, local-safe adaptive budget, rho-threshold sensitivity, and budget target sensitivity.
- Behavioral analysis: how often agents act, whether interventions are local, and whether high survival comes with excessive topology churn.
- Generalization: seen versus unseen chronics and robustness across seeds.

Figures and tables to prepare.

- Table of run families and what each ablation changes.
- Survival curves for paper baseline, GNN, MLP baseline, and sparse-control comparisons.
- Action-frequency plots: action 0 fraction, any non-idle action, and multi-agent simultaneous interventions.
- Summary table with final survival, peak survival, and action sparsity.

Chapter 6: Discussion

Goal. Interpret the results rather than just restating them.

Planned content.

- Answer each research question directly.
- Explain which components appear essential, useful, or unnecessary.

- Discuss the tradeoff between survival and sparse interventions.
- Relate learned behavior to operator-like control: act rarely, act locally, and act when the grid is unsafe.
- Discuss limits: simulator scope, incomplete operational constraints, seed variance, training cost, and dependency on chosen chronics.

Chapter 7: Conclusion and Future Work

Goal. Close with the main thesis claim and next research directions.

Planned content.

- Short summary of the problem, approach, and core findings.
- Final conclusion about decentralized MARL for topology control.
- Future work: richer graph representations, larger grids, outage contingencies, constrained RL tuning, better coordination mechanisms, and operational validation.

1.7 Appendix Plan

- Appendix A: Environment details and action-space construction.
- Appendix B: Hyperparameters and training configuration tables.
- Appendix C: Full ablation run registry.
- Appendix D: Additional survival and behavior plots.
- Appendix E: Reproducibility notes for notebooks, cached W&B histories, and evaluation scripts.

1.8 Immediate Writing Order

1. Write Chapter 3 first, because it fixes notation and the experiment vocabulary.
2. Write Chapter 4 next, using the existing configuration READMEs as source notes for method variants.
3. Draft the Chapter 5 table of ablations before prose, so the results chapter has a stable spine.
4. Write Chapter 1 once the main conclusion is clearer.
5. Finish Chapters 6 and 7 after the final plots and summary metrics are frozen.

1.9 Open Decisions

- Confirm the final thesis title and whether to emphasize GNNs, sparse-control, or decentralized MARL most strongly.
- Decide which experiment families are final and which are only exploratory.
- Decide whether the main claim is performance improvement, improved sparsity at equal survival, or reproducible ablation insight.
- Choose the final bibliography style required by the university.

Chapter 2

Establishing a Baseline

This chapter covers the first stage of the work: obtaining a decentralized MAPPO agent that trains stably on the IEEE 14-bus task, and then testing whether replacing its flat MLP actor with a graph encoder is worth the added complexity. This investigation is motivated by the fact that an MLP's architecture is tied to the specific size of the power grid and thus lacks transferability, whereas a GNN encoder can process graphs of arbitrary size and could theoretically transfer across different grids. Everything here maximizes survival; the representation is fixed and deliberately simple.

Its conclusion sets up the two chapters that follow. The first objective is to establish a strong MLP baseline. We then implement a GNN encoder to determine if it can improve upon, or at least match, the MLP's performance, an important milestone given the GNN's potential for transferability. If a graph encoder proves viable, the next question is which graph, which is what Chapter 3 defines and Chapter 4 screens.

2.1 How to Read This Chapter

This chapter is organized as a sequence of experiment blocks, and the later experimental chapters reuse the same pattern. Each block follows the same structure:

1. **Question.** What uncertainty or design choice does this test address?
2. **Baseline and change.** Which run is the control, and what is changed in the comparison runs?
3. **Protocol.** Which data split, checkpoint, seeds, and metric are used?
4. **Implementation detail.** Which algorithmic, architectural, or mathematical change is being tested, when this is relevant?
5. **Evidence.** Which table and figure summarize the result?
6. **Interpretation.** What can be concluded, and what still needs to be verified?

2.2 Evaluation Protocol

The experiments use a train/test chronic split. The training set contains 80% of the chronics and is used to update the policy. The test set contains the remaining 20% and is used to estimate generalization during training and for final evaluation.

During training, evaluation rollouts are periodically run on both the train and test splits using 10 episodes each time. The main reported metric is the test episodic survival, expressed as a percentage. Higher survival means that the agent keeps the grid alive for a larger fraction of the evaluation episodes. The final plots show seed-aggregated mean curves, faint individual seed curves, and the variability band. The final evaluation currently uses the saved checkpoint with the best test survival.

2.3 Experiment Index

Table 2.1: Current result blocks and their role in the thesis.

Block	Main question	Primary evidence
MLP baseline reruns	Which baseline training changes stabilize MAPPO?	Table 2.4, Figures 2.2 and 2.1
Initial do-nothing bias	Does biasing initial exploration toward do-nothing help?	Table 2.5, Figure 2.3
GINE reruns	Do graph encoders improve survival over the optimized MLP baseline?	Tables C.1 and 2.6, Figure 2.4
Heavy GINE	Does a larger GINE architecture help or overcomplicate optimization?	Table C.2, Figure 2.5

2.4 IEEE 14-Bus Experiments

The first result group focuses on the IEEE 14-bus topology-control task. The initial PPO/MAPPO baseline from the MARL2Grid framework was not satisfactory: the agents did not learn a robust policy and the survival rate remained low. The following experiments therefore test training and architecture changes that could make the baseline more stable.

2.4.1 MLP Baseline Reruns

Question. Which changes are needed to obtain a stable MLP MAPPO baseline on the 14-bus task?

Baseline and changes. We start from the MLP MAPPO baseline of the MARL2Grid framework and test several main stabilization choices:

- increasing actor and critic capacity from two hidden layers to three hidden layers;
- using reward normalization;
- changing the critic update so that the critic is updated once for the whole batch instead of once per agent;
- tuning of optimization hyperparameters to stabilize the training, as summarized in Table 2.2.

Table 2.2: New hyperparameters for MAPPO training.

Hyperparameter	New Value	Paper Baseline
ENVIRONMENT		
n_envs (number parallel environment)	20	10
PPO OPTIMIZATION		
Total time step	20,000,000	25,000,000
Actor learning rate	1e-4	3e-5
Critic learning rate	1e-4	3e-5
Gamma	0.9	0.99
Update epochs	10	80
Minibatches	16	4
Maximum gradient norm	1.0	10.0

Evidence. Table 2.4 reports final and peak survival for the core ablations. Figure 2.2 shows each comparison against the `Optimized MLP Baseline`, and Figure 2.1 overlays all core MLP baseline curves.

Interpretation. The optimized baseline, the non-optimized critic-update variant, and the smaller MLP variant all finish around 93% survival. This suggests that the main recipe is robust once the rerun protocol is fixed. Disabling reward normalization still reaches high peak survival, but it learns more slowly and finishes lower. The old no-validation, non-optimized baseline is clearly not competitive and should not be used as the main reference for later ablations. Because `Optimized MLP Baseline` achieves the highest and most stable final survival, it is adopted as our new standard MLP baseline for all subsequent comparisons.

Table 2.3: MLP baseline run configurations. The reference run is `Optimized MLP Baseline`.

Run family	Actor/critic hidden layers	Reward norm.	Initial action-0 prob.	Optimized critic update	Difference from `Optimized MLP Baseline`
`Optimized MLP Baseline`	3×256 / 3×256	true	0.0	true	Reference baseline
`Disable optimized critic update`	3×256 / 3×256	true	0.0	false	Disables optimized critic updates
`Smaller MLP actor/critic`	2×256 / 2×256	true	0.0	true	Uses a smaller MLP actor and critic
`Disable reward normalization`	3×256 / 3×256	false	0.0	true	Disables reward normalization
`Old baseline (no val)`	3×256 / 3×256	true	0.3	false	Uses action-0 bias 0.3 and non-optimized critic updates
`Initial Bias 50%`	3×256 / 3×256	true	0.5	true	Uses action-0 bias 0.5
`Initial Bias 70%`	3×256 / 3×256	true	0.7	true	Uses action-0 bias 0.7

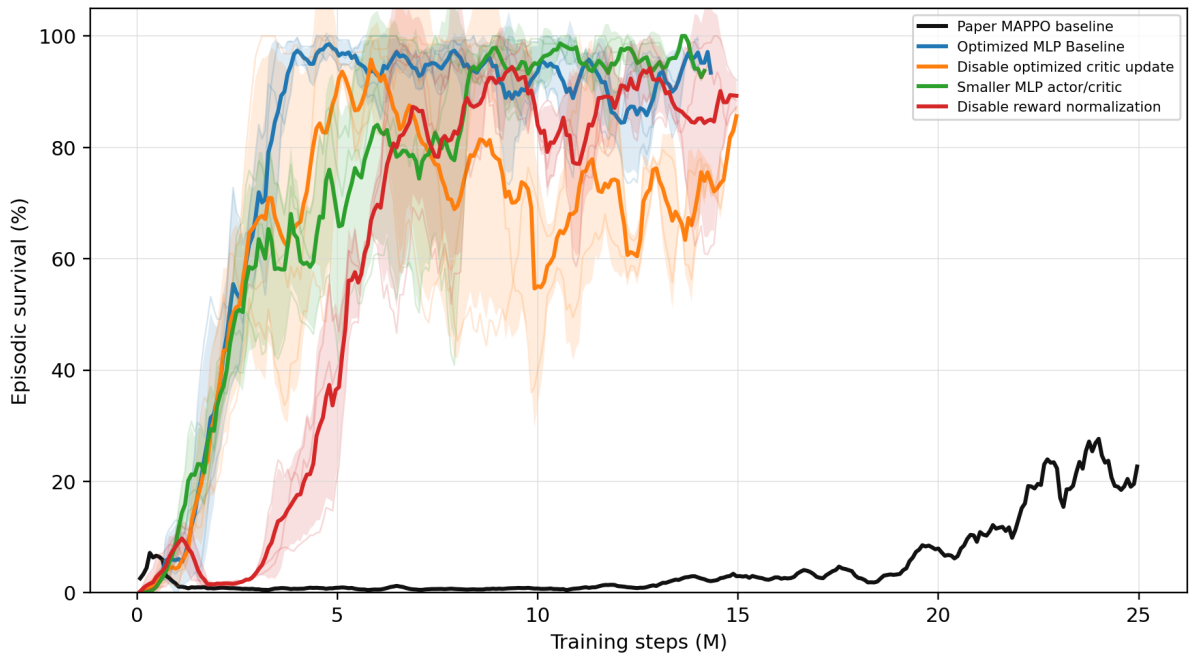


Figure 2.1: All MLP baseline core rerun survival curves shown on one axis.

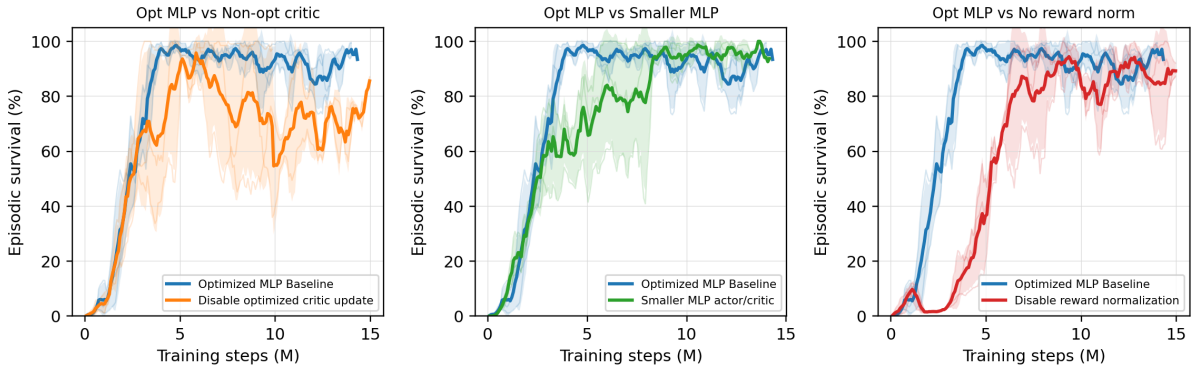


Figure 2.2: Pairwise MLP baseline rerun comparisons against the `Optimized MLP Baseline`.

Table 2.4: MLP baseline core rerun survival summary.

Run	Intended change	Final survival (%)	Peak survival (%)
`Optimized MLP Baseline`	Optimized MLP baseline	93.4	98.6
`Disable optimized critic update`	Non-optimized critic update	93.1	95.8
`Smaller MLP actor/critic`	Smaller MLP architecture	93.7	100.0
`Disable reward normalization`	No reward normalization	81.8	97.1

I need to run a full test eval on the best checkpoint of that MLP baseline

2.4.2 Initial Do-Nothing Bias

Question. Does biasing the initial policy toward action 0, the do-nothing action, help or hurt learning? In theory, a do-nothing bias makes sense because in reality, human operators most often do not operate the grid (i.e., they take no action) under normal conditions.

Baseline and changes. Our baseline uses no initial do-nothing bias in this rerun set. The bias controls keep the same general protocol but change the initial probability mass assigned to action 0.

Implementation detail. The bias is applied only at initialization: it changes the starting action distribution before learning, without changing the reward, architecture, or PPO objective. This is implemented by artificially increasing the initial pre-softmax logits for the do-nothing action, such that the policy initially selects this safe action with a higher probability before any learning updates occur. This makes the ablation a test of exploration bias rather than a test of representational capacity.

Evidence. Table 2.5 and Figure 2.3 compare the zero-bias baseline with `Initial Bias 50%` and `Initial Bias 70%`. The bias values tested are 0.5 and 0.7, corresponding to a 50% and 70% initial probability assigned to the do-nothing action, respectively.

Interpretation. The effect of the action-0 bias is large and non-monotonic. The 0.5-bias curve remains unstable and substantially below the zero-bias baseline, whereas the 0.7-bias curve eventually reaches the highest final survival in this comparison. The current conclusion is that adding an initialization bias is generally hurtful or highly unstable for the models. Instead of aiding exploration, a strong initialization prior appears to restrict the agent's ability to reliably discover diverse beneficial actions and should be treated cautiously as a first-order ablation variable.

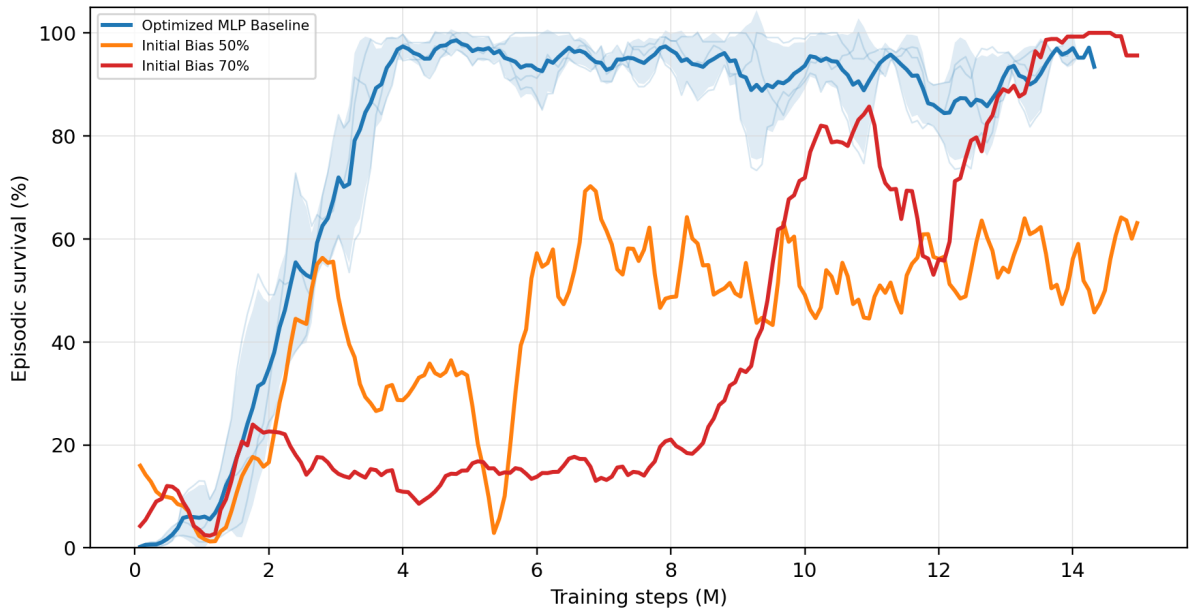


Figure 2.3: Optimized MLP baseline compared with initial do-nothing bias controls `Initial Bias 50%` and `Initial Bias 70%`.

Table 2.5: Initial do-nothing bias survival summary.

Run	Final survival (%)	Peak survival (%)
`Optimized MLP Baseline`	93.4	98.6
`Initial Bias 50%`	50.8	70.2
`Initial Bias 70%`	99.3	100.0

I might need to run a full test eval of these runs maybe i need to rerrun all these runs because they don't go/stop at 15M steps

2.4.3 GNN Encoder Viability and Architecture

Question. Can a graph encoder perform as well as the MLP? This is a crucial question because if all agents share the same GNN encoder, the architecture becomes independent of the grid size and could maybe transfer to any grid topology. If each agent requires its own separate encoder or if the GNN cannot stabilize, this transferability is lost.

Baseline and changes. We conducted an exploratory study comparing several variations of the GINE architecture. We tested the encoder size (light vs. heavy GINE), actor sharing (shared vs. unshared encoder), the critic architecture (MLP vs. GNN), and the impact of the optimizations we established for the MLP baseline (optimized critic updates, reward normalization, and removing the initial do-nothing bias).

Protocol. All GINE runs use `gnn_type = gine`, 15,000,000 total timesteps, evaluation every 80,000 timesteps, 2,000 rollout steps, deterministic evaluation, and the same 80/20 chronic split used by the MLP experiments. We use GINE because it naturally incorporates edge features and has been shown to be powerful at distinguishing graph topologies [1]. The complete graph-actor pipeline is described in Appendix A, and Appendix C details the exhaustive parameter configurations used for these ablations in Tables C.1 and C.2.

Evidence. Table 2.6 and Figure 2.4 compare the core architectural improvements against the `Heavy GINE Baseline`. Figure 2.5 provides a broader search, showing how the baseline reacts to individual

isolated changes.

Table 2.6: GINE rerun survival summary.

Run	Intended change	Final survival (%)	Peak survival (%)
`Heavy GINE Baseline`	Historical GINE baseline	59.2	93.9
`Light GINE (Concat, MLP Critic)`	Light GINE with MLP critic	78.9	90.4
`No Bias, Opt Critic`	Optimized critic and bias 0.0	82.5	85.7
`Optimized Light GINE`	Light optimized GINE with MLP critic	95.7	97.4

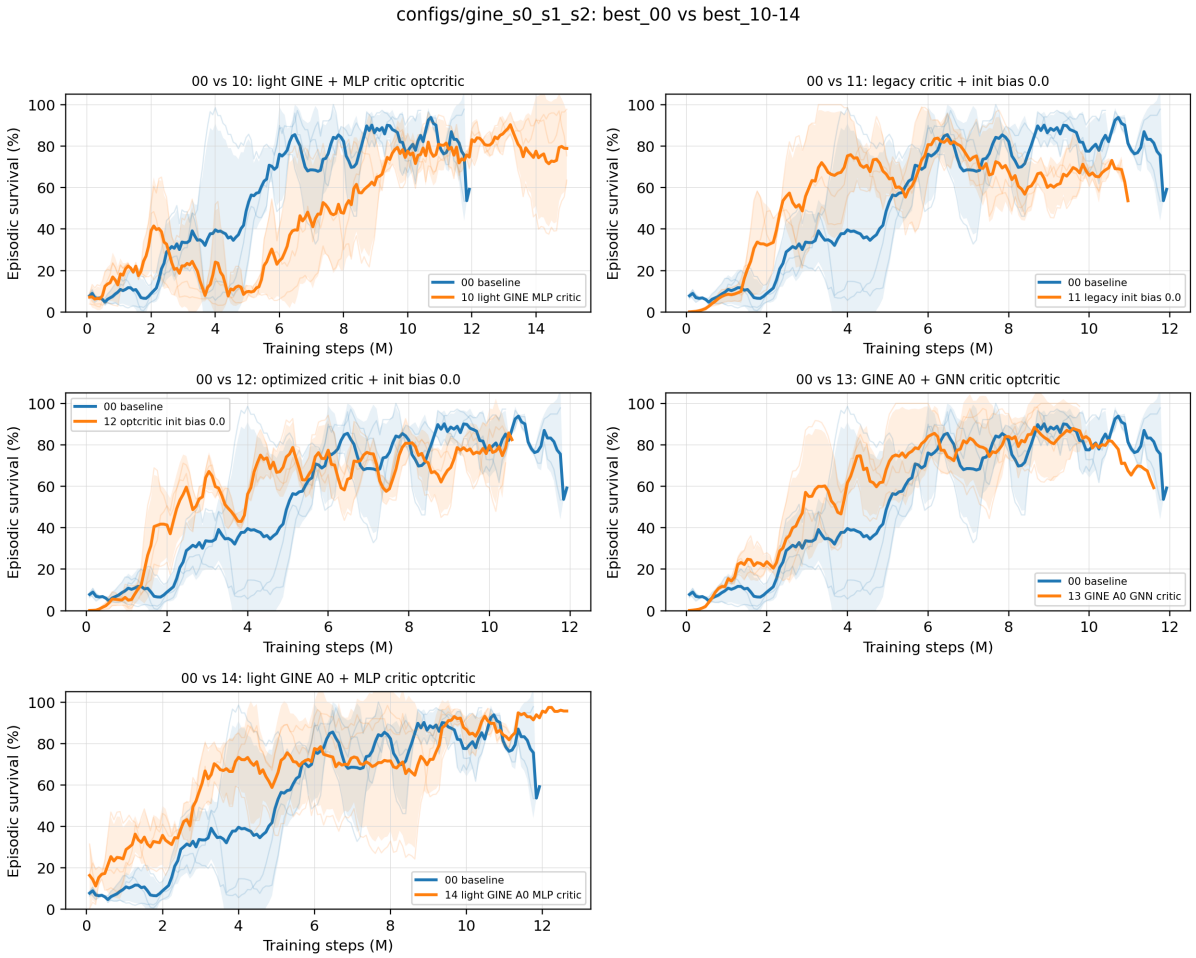


Figure 2.4: Seed-aggregated test episodic survival for the `Heavy GINE Baseline` against various architectural modifications.

Interpretation. The results from the broad search and the core reruns provide several critical takeaways regarding the GNN architecture:

- **Shared Encoder Viability:** As shown in Figure 2.5, the `Unshared Actor` variant does not outperform the `Heavy GINE Baseline`. This proves that a single shared GNN encoder for all agents is achievable and does not bottleneck performance, which is exactly what is needed for transferability.
- **Critic Architecture:** Attempting to use a GNN for the centralized critic proved unstable and degraded performance. The `MLP Critic` variants were significantly more stable. The best performance is clearly achieved by pairing the shared GNN actor with a standard MLP critic.
- **Optimization Synergies:** The GNN architecture is heavily dependent on the same optimizations that stabilized our MLP baseline. Specifically, optimized critic updates, reward normalization, and

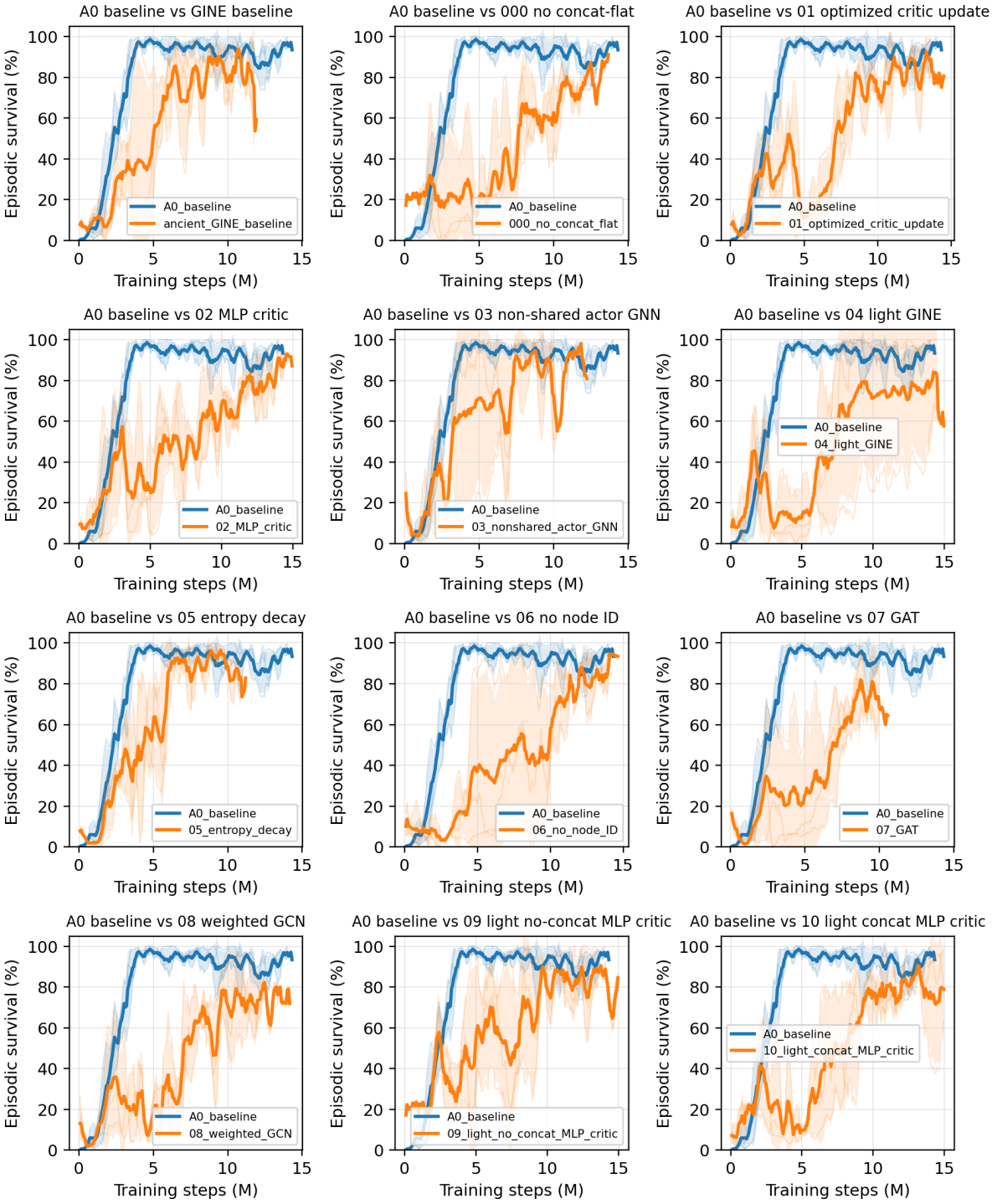


Figure 2.5: Heavy-GINE survival comparisons. Each subplot compares the `Heavy GINE Baseline` (heavy concat-flat GINE) against one controlled alternative from `configs/gine_s0_s1_s2`.

removing the initial do-nothing bias (bias 0.0) were crucial for getting the GNN to learn reliably. The strongest overall result is the `Optimized Light GINE`, which combines the shared light GINE actor, the MLP critic, and all the baseline optimizations to reach 95.7% final survival. This validates that a GNN encoder is highly competitive on the 14-bus task.

With a stable, viable, and theoretically transferable GNN architecture established, the next critical question is which graph representation best captures the underlying power grid. This sets the stage for the representational screening in Chapter 3.

Chapter 3

Methods

This chapter describes the graph inputs and graph-actor variants used in the experiments. The presentation separates four design choices: feature scaling, the base graph representation, substation-level augmentations, and graph pooling.

3.1 Scaling and Normalization

Two independent transformations can be applied to the continuous node and edge features of every graph schema: deterministic physical scaling followed by optional running standardization. Let $x_{i,f,t}$ be feature f of node or edge i at time t. The complete pipeline is

$$x^{\text{scale}}_{i,f,t} = \begin{cases} x_{i,f,t}/s_f, & \text{if physical scaling is enabled,} \\ x_{i,f,t}, & \text{otherwise,} \end{cases} \tag{3.1}$$

$$\widehat{x}_{i,f,t} = \begin{cases} \operatorname{clip}\left(\dfrac{x^{\text{scale}}_{i,f,t} - \mu_{f,t}}{\sqrt{\sigma^2_{f,t} + \epsilon}}, -c, c \right), & \text{if running normalization is enabled,} \\ x^{\text{scale}}_{i,f,t}, & \text{otherwise.} \end{cases} \tag{3.2}$$

Here s_f is a fixed physical reference and $\mu_{f,t}$ and $\sigma^2_{f,t}$ are running moments estimated during training. When both transformations are enabled, the moments describe x/s_f. Clipping applies only to running standardization; $c = 10$ by default and a non-positive value disables clipping.

3.1.1 Deterministic physical scaling

Physical scaling divides each quantity by a constant with the same unit and therefore preserves physical zero. Table 3.1 gives the exact constants. The power base may be specified explicitly; otherwise the largest installed generator rating is used, with a 100 MW fallback when the environment does not expose generator ratings.

Table 3.1: Deterministic physical scales used for continuous graph features.

Features	Scale s_f	Source
`gen_p`, `load_p`, `p`	Power base in MW	Configured value or maximum generator rating.
`gen_theta`, `load_theta`, `theta`	180 degrees	Fixed angular scale.
`time_before_cooldown_sub`	Maximum substation cooldown	Grid2Op environment parameter.
`time_before_cooldown_line`	Maximum line cooldown	Grid2Op environment parameter.
`timestep_overflow`	Allowed overflow duration	Grid2Op environment parameter.
Maintenance time and duration	Episode horizon	Grid2Op chronic horizon.

3.1.2 Running standardization

Running moments are estimated with Welford's online algorithm. One normalizer is shared across agents and updated from the full state graph once per environment step. Heterogeneous graphs maintain separate node statistics for each node type. Edge statistics include only active physical-line relations; inactive candidates and structural, asset-attachment, same-substation, and virtual relations do not contribute.

The loading ratio `rho` is already dimensionless and is neither rescaled nor standardized. Preserving it is also necessary when GCN uses `rho` directly as a non-negative message weight. Binary connection states, node and edge masks, domain masks, type indicators, and relation one-hot attributes are also left unchanged. Only the continuous channels listed in Table 3.1 enter the running statistics. The running moments are frozen during evaluation.

3.2 Graph Representations

The graph observation can use one of three base representations. The `bus` representation aggregates equipment values onto busbar nodes. The `heterogeneous` representation disaggregates generators and loads into explicit asset nodes while keeping physical lines as busbar-to-busbar edges. The `heterogeneous_line` representation further disaggregates transmission lines into explicit line nodes.

Table 3.2 states what separates them. The three differ in one respect only, how much of the grid is given its own node, and everything below follows from that choice.

Table 3.2: The three graph schemas. Each row is a consequence of how much equipment receives its own node.

Aspect	`bus`	`heterogeneous`	`heterogeneous_line`
Node types	Busbar	Busbar, generator, load	Busbar, generator, load, transmission line
Generator and load state	Summed and averaged onto the busbar	One node per asset	One node per asset
Physical line	Direct busbar-to-busbar edge	Typed direct busbar-to-busbar edge	Typed node between its endpoint busbars
Line state	Continuous edge attributes	Continuous edge attributes	Features of the line node
Endpoint path	One hop between busbars	One hop between busbars	Two hops through the line node
Minimum useful depth	One layer for adjacent busbar exchange	One layer for adjacent busbars; two for asset-to-remote-busbar flow	Two layers for adjacent busbar exchange through a line node
Hidden line state	None: the edge modifies a message	None: the edge modifies a message	Yes: each line obtains and updates an embedding
Node count	SB	$SB + N_g + N_d$	$SB + N_g + N_d + L$

3.2.1 Candidate graph and dynamic mask

All three schemas share one construction.

Grid2Op exposes the instantaneous electrical state and topology through a structured observation [2]. For each representation, the graph construction keeps the set of candidate nodes and edges fixed throughout an episode while using a dynamic edge mask to express the topology that is electrically active at time step t.

For agent a, the graph observation can be written as

$$\mathcal{G}_t^{(a)} = \left(\mathcal{V}^{(a)}, \mathcal{E}_{\text{cand}}^{(a)}, \mathbf{X}_t^{(a)}, \mathbf{Z}_t^{(a)}, \mathbf{m}_t^{(a)}\right), \tag{3.3}$$

where $\mathcal{V}^{(a)}$ and $\mathcal{E}_{\text{cand}}^{(a)}$ are static, $\mathbf{X}_t^{(a)}$ contains node features, $\mathbf{Z}_t^{(a)}$ contains edge features, and $\mathbf{m}_t^{(a)}$ indicates which candidate edges are active. This separates the physical equipment layout, which does not change during an episode, from the busbar assignment and line status, which can change after every topology action.

The mechanism is easiest to see on a transmission line, which is where every schema uses it. Consider a physical line ℓ whose origin and extremity belong to substations $o(\ell)$ and $e(\ell)$. For every pair of possible endpoint busbars (b_o, b_e), the graph reserves the two directed candidate edges

$$i(o(\ell), b_o) \to i(e(\ell), b_e) \quad \text{and} \quad i(e(\ell), b_e) \to i(o(\ell), b_o), \tag{3.4}$$

so each physical line contributes $2B^2$ candidate directed edges and the two directions share the same line attributes. For the usual case $B = 2$, one line reserves eight candidate edges even though at most two, one per direction, are electrically active at any time.

Which of them is active is decided by the mask

$$m_{\ell,b_o,b_e,t} = \mathbf{1}[\text{line } \ell \text{ is connected at } t]\, \mathbf{1}\left[b^{\text{or}}_{\ell,t} = b_o\right] \mathbf{1}\left[b^{\text{ex}}_{\ell,t} = b_e\right]. \tag{3.5}$$

A disconnected line has all of its candidates masked; a connected one keeps only the pair matching its current origin and extremity busbars. A topology action therefore changes connectivity by changing $\mathbf{m}_t^{(a)}$, never by rebuilding the graph, and the node and edge tensors keep a fixed shape for the whole episode. The same pattern governs the asset attachments of Section 3.2.3 and the line-node attachments of Section 3.2.4; Appendix B gives the resulting sizes and the masking conventions.

The following subsections define the three schemas.

3.2.2 Busbar Representation

The `bus` schema is the compact busbar representation. Busbars are the only nodes, active transmission lines form the graph backbone, and generator and load quantities are aggregated into their currently assigned busbars.

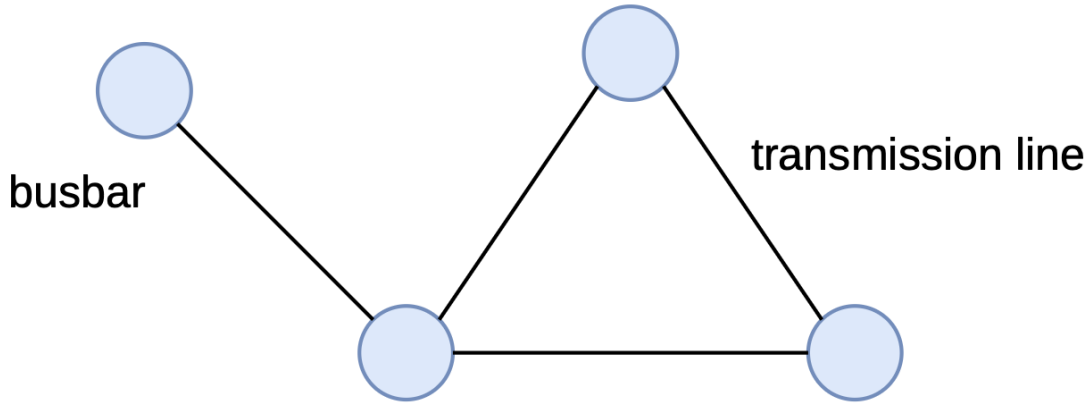


Figure 3.1: Busbar graph representation. Blue nodes denote busbars and black connections denote active transmission-line relations. Generator and load quantities are aggregated into the features of their associated busbars.

Nodes and features

Let S denote the number of substations and let B denote the maximum number of busbars per substation. A node is created for every substation–busbar pair, including busbars to which no asset is connected at the current time. A global node identifier is assigned as

$$i(s, b) = sB + b, \qquad s \in \{0, \dots, S-1\}, \quad b \in \{0, \dots, B-1\}. \tag{3.6}$$

The complete graph therefore contains SB nodes.

Table 3.3 lists the six node features. Generator and load quantities are assigned to the busbar specified

by the current topology vector. Active powers are summed when several assets occupy the same busbar, whereas the corresponding angles are averaged. Disconnected assets do not contribute to any node. The substation cooldown is repeated on every busbar node of that substation.

Table 3.3: Features attached to each busbar node.

Feature	Definition
`gen_p`	Sum of active generator power connected to the busbar.
`gen_theta`	Mean voltage angle of generators connected to the busbar.
`load_p`	Sum of active load power connected to the busbar.
`load_theta`	Mean voltage angle of loads connected to the busbar.
`time_before_cooldown_sub`	Remaining substation cooldown.
`domain_mask`	One for a substation controlled by the agent and zero for a contextual neighbor.

This aggregation produces a compact representation but deliberately removes the identities and counts of individual generators and loads.

Relations and edge features

The busbar schema instantiates Equation 3.4 directly: its only physical relations are the busbar-to-busbar candidates of a transmission line. The dynamic attributes copied onto active physical-line edges are given in Table 3.4. The two maintenance features are present only for environments with scheduled maintenance.

Table 3.4: Features attached to physical-line edges.

Feature	Definition
`line_status`	Binary connection status of the physical line.
`rho`	Line loading relative to its thermal limit.
`timestep_overflow`	Number of consecutive time steps in overload.
`time_before_cooldown_line`	Remaining cooldown before another line action is legal.
`time_next_maintenance`	Time until scheduled maintenance, when available.
`duration_next_maintenance`	Duration of scheduled maintenance, when available.

For GINE and GAT, the edge embedding of a busbar-to-busbar transmission edge is therefore exactly the row of Table 3.4. Optional self and same-substation edges use the same feature width: all physical channels are zero except `rho`$= 1$, which is a neutral structural value. The homogeneous schema does not append relation one-hots, so these optional relations are distinguished only by their endpoints and neutral physical attributes.

Message-passing path

Asset values already sit on their busbar, so information follows the single path busbar $\leftrightarrow$ busbar and one message-passing layer is enough to carry a substation's state to an adjacent one. This is the shortest path of the three schemas, and the baseline against which the two disaggregated ones trade extra hops for individually addressable equipment.

3.2.3 Disaggregated Representation

The `heterogeneous` schema keeps busbars as the backbone and adds one node per generator and load. Transmission lines remain busbar-to-busbar edges. The node set is

$$\mathcal{V} = \mathcal{V}_{\text{bus}} \cup \mathcal{V}_{\text{gen}} \cup \mathcal{V}_{\text{load}}, \tag{3.7}$$

with typed directed relations between these sets. Generator and load states are therefore preserved individually.

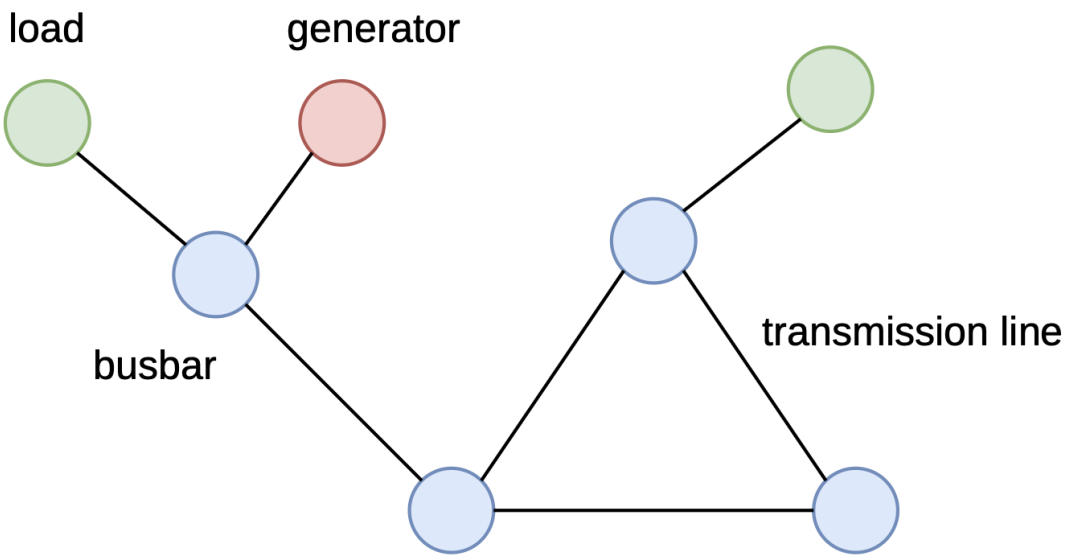


Figure 3.2: Heterogeneous equipment representation. Busbars are blue, generators are red, and loads are green. Physical transmission lines remain direct relations between busbar nodes.

Mean readout includes all nodes in the local graph, including one-hop context.

Nodes and features

For a graph with S included substations, B busbars per substation, N_g included generators, and N_d included loads, the number of nodes is

$$|\mathcal{V}| = SB + N_g + N_d. \tag{3.8}$$

Busbar nodes are created exactly as in Section 3.2.2. In addition, each generator and load whose substation is present in the graph receives its own node.

Every busbar, generator, and load node is represented by the same ordered eight-dimensional feature vector

$$\begin{aligned}\mathbf{x}_v = [&\texttt{is\_busbar},\ \texttt{is\_generator},\ \texttt{is\_load},\ \texttt{p},\ \texttt{theta},\\ &\texttt{time\_before\_cooldown\_sub},\ \texttt{connected},\ \texttt{domain\_mask}]^\top \in \mathbb{R}^8.\end{aligned} \tag{3.9}$$

The width, column order, and meaning of this vector are identical for all node types; node type does not select a smaller feature vector. Table 3.5 shows the value placed in every shared column. A zero in the table means that the column is still present for that node type but is explicitly zero-filled.

Table 3.5: Values placed in the eight feature columns shared by every node in the direct-line heterogeneous graph. Disconnected assets retain their node row but have zero power and angle.

Shared feature column	Busbar node	Generator node	Load node
`is_busbar`	1	0	0
`is_generator`	0	1	0
`is_load`	0	0	1
`p`	0	`gen_p` if connected; 0 otherwise	`load_p` if connected; 0 otherwise
`theta`	0	`gen_theta` if connected; 0 otherwise	`load_theta` if connected; 0 otherwise
`time_before_cooldown_sub`	Substation value	0	0
`connected`	1	1 connected; 0 disconnected	1 connected; 0 disconnected
`domain_mask`	1 controlled; 0 contextual	1 controlled; 0 contextual	1 controlled; 0 contextual

Relations and edge features

Physical lines keep the busbar-to-busbar candidates of Equation 3.4; what this schema adds is the asset attachments. For generator g at substation $s(g)$, the fixed candidate graph contains

$$g \to i(s(g), b) \quad \text{and} \quad i(s(g), b) \to g, \qquad b \in \{0, \dots, B-1\}, \tag{3.10}$$

that is, B candidates per direction. Loads follow the same rule. Appendix B collects the resulting candidate-edge counts.

Every candidate directed edge is represented by the same ordered feature vector. Its physical block is

$$\begin{aligned} \mathbf{p}_{uv} = [\,&\texttt{line\_status}, \texttt{rho}, \texttt{timestep\_overflow}, \\ &\texttt{time\_before\_cooldown\_line}, \\ &(\texttt{time\_next\_maintenance}, \texttt{duration\_next\_maintenance})_{\text{optional}}\,]^{\mathsf{T}}. \end{aligned} \tag{3.11}$$

The two maintenance columns are appended only when scheduled maintenance is available. The relation block is the same seven-column vector for every edge:

$$\begin{aligned} \mathbf{r}_{uv} = [\,&\texttt{relation\_self}, \texttt{relation\_physical\_line}, \\ &\texttt{relation\_same\_substation\_busbar}, \\ &\texttt{relation\_generator\_to\_busbar}, \texttt{relation\_busbar\_to\_generator}, \\ &\texttt{relation\_load\_to\_busbar}, \texttt{relation\_busbar\_to\_load}\,]^{\mathsf{T}}. \end{aligned} \tag{3.12}$$

The complete common edge vector is therefore

$$\mathbf{e}_{uv} = [\,\mathbf{p}_{uv}^{\mathsf{T}} \mid \mathbf{r}_{uv}^{\mathsf{T}}\,]^{\mathsf{T}} \in \begin{cases} \mathbb{R}^{11}, & \text{without maintenance columns,} \\ \mathbb{R}^{13}, & \text{with maintenance columns.} \end{cases} \tag{3.13}$$

Its width, column order, and meaning do not change with the relation type. An active physical-line edge fills $\mathbf{p}_{uv}$ with its measured line state. Every active structural or asset-attachment edge instead uses

$$\mathbf{p}_{\text{neutral}} = [\,0, 1, 0, 0, (0, 0)_{\text{optional}}\,]^{\mathsf{T}}, \tag{3.14}$$

so only the shared `rho` column is nonzero. In every active edge row, exactly one column of $\mathbf{r}_{uv}$ is one and the other six are zero.

Table 3.6 shows the values placed in these common blocks for every relation.

Table 3.6: Values placed in the common edge-feature vector shared by every active directed relation in the direct-line heterogeneous graph. "Other six" refers to the remaining columns of the seven-column relation block.

Directed relation	Active rule	Shared physical block	Shared relation block
$v \to v$ (self)	Optional; always active when enabled	$\mathbf{p}_{\text{neutral}}$	`relation_self`= 1; other six = 0
$i_o \to i_e$ and reverse (physical line)	Line online and candidate busbars match the current endpoints	Measured $\mathbf{p}_{uv}$	`relation_physical_line`= 1; other six = 0
$i(s,b) \to i(s,b')$, $b \neq b'$	Optional; every ordered same-substation busbar pair	$\mathbf{p}_{\text{neutral}}$	`relation_same_substation_busbar`= 1; other six = 0
$g \to i(s(g), b)$	Generator connected to candidate busbar b; direction enabled	$\mathbf{p}_{\text{neutral}}$	`relation_generator_to_busbar`= 1; other six = 0
$i(s(g), b) \to g$	Same mask; reverse direction enabled	$\mathbf{p}_{\text{neutral}}$	`relation_busbar_to_generator`= 1; other six = 0
$d \to i(s(d), b)$	Load connected to candidate busbar b; direction enabled	$\mathbf{p}_{\text{neutral}}$	`relation_load_to_busbar`= 1; other six = 0
$i(s(d), b) \to d$	Same mask; reverse direction enabled	$\mathbf{p}_{\text{neutral}}$	`relation_busbar_to_load`= 1; other six = 0

Generator and load directions are selected independently as `bidirectional`, `asset_to_busbar`, or `busbar_to_asset`. The default is `bidirectional`. A one-way mode retains only one member of each pair in Equation 3.10. Physical-line and same-substation relations remain bidirectional.

Message-passing path

With bidirectional asset and line relations, information follows the path

$$\text{generator} \leftrightarrow \text{local busbar} \leftrightarrow \text{remote busbar} \leftrightarrow \text{load}. \tag{3.15}$$

One message-passing layer crosses one arrow. Generator or load information therefore reaches its local busbar after one layer and the adjacent remote busbar after two. With one-way asset relations, the reverse flow is absent.

3.2.4 Disaggregated Representation with Line Nodes

The `heterogeneous_line` schema replaces each direct physical-line edge with a transmission-line node between its endpoint busbars. Its node set is

$$\mathcal{V} = \mathcal{V}_{\text{bus}} \cup \mathcal{V}_{\text{gen}} \cup \mathcal{V}_{\text{load}} \cup \mathcal{V}_{\text{line}}. \tag{3.16}$$

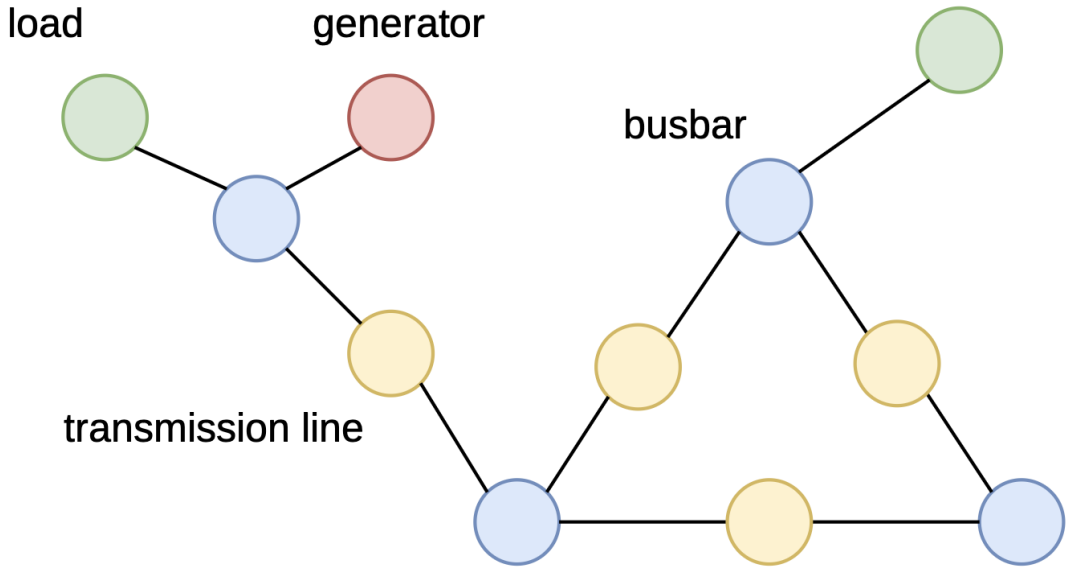


Figure 3.3: Heterogeneous representation with explicit transmission-line nodes. Yellow nodes represent physical lines and connect the blue busbars at their endpoints; generators and loads remain explicit terminal nodes.

Nodes and features

For S included substations, B busbars per substation, N_g generators, N_d loads, and L included transmission lines, the graph contains

$$|\mathcal{V}| = SB + N_g + N_d + L \tag{3.17}$$

nodes.

Every node type uses the same ordered feature vector:

$$\begin{aligned}\mathbf{x}_v = [&\texttt{is\_busbar}, \texttt{is\_generator}, \texttt{is\_load}, \texttt{is\_transmission\_line},\\ &\texttt{p}, \texttt{theta}, \texttt{line\_status}, \texttt{rho}, \texttt{timestep\_overflow},\\ &\texttt{time\_before\_cooldown\_line},\\ &(\texttt{time\_next\_maintenance}, \texttt{duration\_next\_maintenance})_{\text{optional}},\\ &\texttt{time\_before\_cooldown\_sub}, \texttt{connected}, \texttt{domain\_mask}]^{\mathsf{T}},\\ \mathbf{x}_v \in &\begin{cases}\mathbb{R}^{13}, & \text{without maintenance columns},\\ \mathbb{R}^{15}, & \text{with maintenance columns}.\end{cases}\end{aligned} \tag{3.18}$$

The width and column order are identical for busbar, generator, load, and transmission-line nodes. Table 3.7 specifies every entry; 0 denotes an explicitly zero-filled column, not an omitted feature. For generators and loads, power and angle are set to zero when the asset is disconnected.

Table 3.7: Values placed in the common node-feature vector of the explicit-line heterogeneous graph. The two maintenance rows are included only when maintenance observations are enabled.

Shared feature column	Busbar	Generator	Load	Transmission line
`is_busbar`	1	0	0	0
`is_generator`	0	1	0	0
`is_load`	0	0	1	0
`is_transmission_line`	0	0	0	1
`p`	0	`gen_p` if connected; 0 otherwise	`load_p` if connected; 0 otherwise	0
`theta`	0	`gen_theta` if connected; 0 otherwise	`load_theta` if connected; 0 otherwise	0
`line_status`	0	0	0	Observed line value
`rho`	0	0	0	Observed ρ_ℓ
`timestep_overflow`	0	0	0	Observed line value
`time_before_cooldown_line`	0	0	0	Observed line value
`time_next_maintenance`	0	0	0	Observed line value
`duration_next_maintenance`	0	0	0	Observed line value
`time_before_cooldown_sub`	Substation value	0	0	0
`connected`	1	1 connected; 0 disconnected	1 connected; 0 disconnected	1 online; 0 offline
`domain_mask`	1 controlled; 0 contextual	1 controlled; 0 contextual	1 controlled; 0 contextual	1 if either endpoint is controlled; 0 otherwise

Relations and edge features

This schema replaces the busbar-to-busbar candidates of Equation 3.4 by attachments to a line node. Let $o(\ell)$ and $e(\ell)$ denote the origin and extremity substations of line ℓ. For every busbar index b, the fixed candidate graph contains

$$\begin{aligned} &i(o(\ell), b) \to \ell, \qquad \ell \to i(o(\ell), b), \\ &i(e(\ell), b) \to \ell, \qquad \ell \to i(e(\ell), b), \end{aligned} \qquad b \in \{0, \dots, B-1\}, \tag{3.19}$$

that is, $2B$ candidates per end.

The line direction is `bidirectional`, `line_to_busbar`, or `busbar_to_line`; a one-way mode retains only one column of Equation 3.19. An online line therefore has four active attachments in the bidirectional case and two in a one-way case.

Every candidate directed edge uses the same ordered ten-dimensional feature vector:

$$\begin{aligned} \mathbf{e}_{uv} = [&\texttt{relation\_self}, \texttt{relation\_same\_substation\_busbar}, \\ &\texttt{relation\_busbar\_to\_line\_origin}, \texttt{relation\_line\_origin\_to\_busbar}, \\ &\texttt{relation\_busbar\_to\_line\_extremity}, \texttt{relation\_line\_extremity\_to\_busbar}, \\ &\texttt{relation\_generator\_to\_busbar}, \texttt{relation\_busbar\_to\_generator}, \\ &\texttt{relation\_load\_to\_busbar}, \texttt{relation\_busbar\_to\_load}]^\top \in \mathbb{R}^{10}. \end{aligned} \tag{3.20}$$

These edges contain no continuous line-state attributes because the transmission-line node already contains `rho`, status, overflow, cooldown, and maintenance. For an active edge, exactly one relation column is one and the other nine are zero. An inactive candidate has all ten entries set to zero and is removed by `edge_mask`$= 0$ before message passing. Table 3.8 gives the active rule and the nonzero entries for every relation.

Table 3.8: Nonzero entries in the common 10-column edge-feature vector of the explicit-line heterogeneous graph. All relation columns not named in a row are zero.

Directed relation	Active rule	Relation column set to 1
$v \to v$ (self)	Optional; always active when enabled	`relation_self`
$i(s,b) \to i(s,b')$, $b \neq b'$	Optional; every ordered same-substation pair	`relation_same_substation_busbar`
$i_o \to \ell$	Online line, current origin busbar, direction enabled	`relation_busbar_to_line_origin`
$\ell \to i_o$	Same mask; reverse direction enabled	`relation_line_origin_to_busbar`
$i_e \to \ell$	Online line, current extremity busbar, direction enabled	`relation_busbar_to_line_extremity`
$\ell \to i_e$	Same mask; reverse direction enabled	`relation_line_extremity_to_busbar`
$g \to i(s(g), b)$	Generator attached to b; direction enabled	`relation_generator_to_busbar`
$i(s(g), b) \to g$	Same mask; reverse direction enabled	`relation_busbar_to_generator`
$d \to i(s(d), b)$	Load attached to b; direction enabled	`relation_load_to_busbar`
$i(s(d), b) \to d$	Same mask; reverse direction enabled	`relation_busbar_to_load`

Origin, extremity, and direction have separate relation types. Generator and load attachments follow Equation 3.10; self and same-substation relations remain optional.

Message-passing path

The direct-line graph connects endpoint busbars in one hop. The explicit-line graph uses

$$i_o \to \ell \to i_e, \tag{3.21}$$

so endpoint exchange requires two layers. The first layer forms

$$\mathbf{h}_\ell^{(1)} = f_\ell\left(\mathbf{h}_\ell^{(0)}, \mathbf{h}_{i_o}^{(0)}, \mathbf{h}_{i_e}^{(0)}\right), \tag{3.22}$$

and the second returns this joint state to the endpoints. Later layers enlarge the receptive field. The edges around the line node are bidirectional: the line gathers both endpoints and sends the updated state back; endpoint exchange is possible.

3.2.5 Local actor graphs and the global state graph

Each agent controls a set of substations $\mathcal{C}_a$. Under decentralized observability, its local graph can be constructed in two ways. Without neighbor context, the node set contains only substations in $\mathcal{C}_a$, and the line set contains only lines whose two endpoints lie inside this region. With one-hop context enabled, the graph additionally contains every line touching $\mathcal{C}_a$ and the substations at the opposite endpoints of those lines. The `domain_mask` node feature and a separate controlled-node mask distinguish the controlled region from contextual neighbors. This second mask also permits graph readout to pool only the nodes on which the agent can act.

The full-grid state graph is available to the centralized critic during training, while each actor receives only its local graph. This follows the centralized-training, decentralized-execution structure of MAPPO [3].

3.2.6 Graph encoder

GINE and GAT receive every column of the edge-feature vector. The homogeneous graph provides only physical-line columns; the heterogeneous graphs append the relation one-hot columns defined in Tables 3.6 and 3.8. Node types are encoded by the explicit `is_*` node features.

GINE and GAT use the same high-level actor pipeline:

$$\begin{aligned}(\mathbf{X}, \mathbf{E}, \mathbf{m}) &\longrightarrow \text{active-edge filtering} \longrightarrow \text{node projection} \longrightarrow \mathbf{H}^{(0)},\\ \mathbf{H}^{(0)} &\longrightarrow K \text{ message-passing layers} \longrightarrow \text{graph readout} \longrightarrow \text{action logits.}\end{aligned} \tag{3.23}$$

They differ only in the message-passing block. GINE inserts the projected edge vector into the content of every message. GAT uses the projected edge vector to compute the attention weight assigned to that

message. Pooling, the graph output projection, and the action head are otherwise identical. The exact equations and architectural dimensions are given in Appendix A.

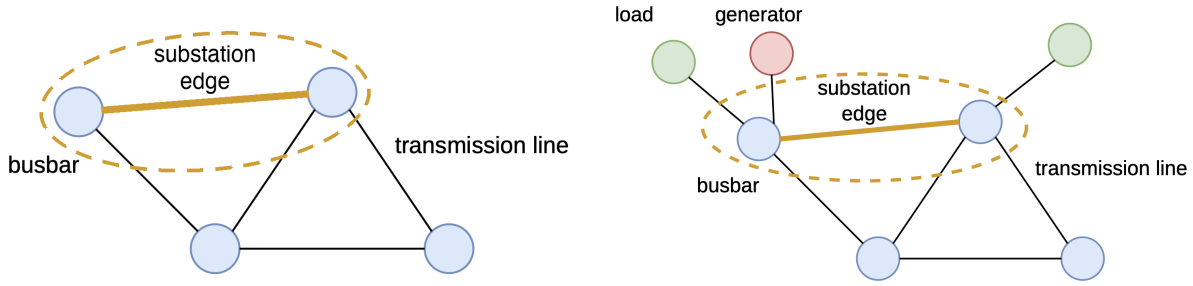
The busbar schema aggregates generator and load states and omits reactive power, voltage magnitudes, and directional endpoint flows. Empty busbars remain in the node set. The preprocessing in Section 3.1 changes feature values but not the graph schema.

3.3 Substation Representation: Direct Edges and Summary Nodes

Two optional factors change how information is exchanged inside a substation. The factor e adds direct same-substation busbar edges. The factor n adds one learned substation-summary node per included substation.

3.3.1 Direct same-substation edges (e)

In addition to physical-line relations, the graph may include a self-edge $v \to v$ for every node and a directed same-substation edge $i(s, b) \to i(s, b')$ for every $b \neq b'$. These relations remain active throughout the episode and use zero physical-line features except for the neutral value `rho`$= 1$. Same-substation edges permit message passing between alternative busbars but do not represent an electrical connection.



(a) Busbar graph.

(b) Disaggregated graph.

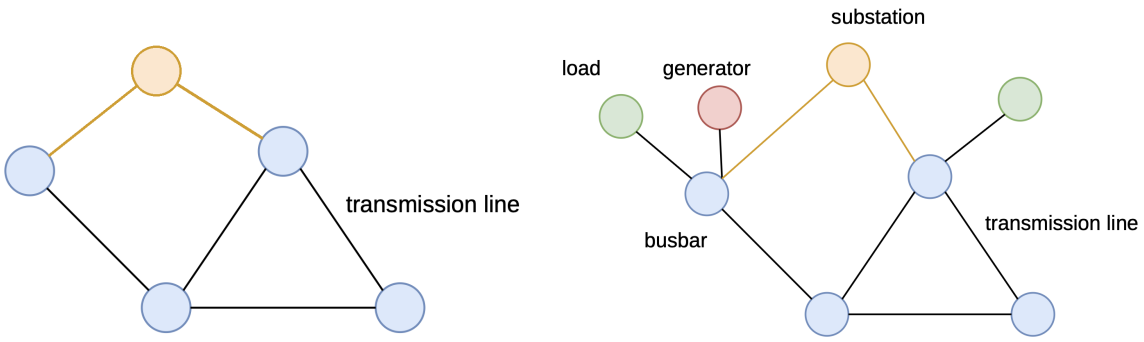
Figure 3.4: Direct same-substation busbar relation, shown in the two schemas of Section 3.2. The highlighted orange edge allows two busbars belonging to the same substation to exchange messages; it is a computational relation and not a physical transmission line. In (a) the generator and load quantities are aggregated into the busbar features, whereas in (b) they are separate nodes. The relation is identical in both cases: it connects two busbars of one substation and does not depend on the node types attached to them.

3.3.2 Substation summary nodes (n)

An optional hierarchical augmentation introduces one learned node u_s for every included substation s. Let $\mathcal{B}(s)$ be its included busbar nodes. Summary edges are either `bidirectional` or `toward_summary`.In the latter case, the augmented graph contains only

$$\mathcal{E}_{\text{sub}} = \{(i, u_s) \mid i \in \mathcal{B}(s)\}\,; \tag{3.24}$$

the bidirectional setting additionally includes every reverse edge (u_s, i). Each substation node therefore gathers information from every busbar belonging to that substation. In the bidirectional case it can also redistribute its learned summary to those busbars in later message-passing layers. It is not connected directly to generators, loads, or transmission-line nodes; those elements influence it through their busbar attachments.



(a) Busbar graph.

(b) Disaggregated graph.

Figure 3.5: Learned substation-summary augmentation, shown in the two schemas of Section 3.2. The orange substation node receives messages from each of its blue busbar nodes and can optionally redistribute the resulting substation-level representation. In (a) the generator and load quantities are aggregated into the busbar features, whereas in (b) they are separate nodes; in both cases the summary node attaches only to busbars, so generators, loads, and transmission-line nodes reach it through their busbar attachments.

The initial substation-node states are learned embeddings indexed by the global substation identifiers. For S included substations and B busbars per substation, the augmentation adds S nodes and either SB directed edges for `toward_summary` or $2SB$ for `bidirectional`. The nodes participate in standard mean, sum, maximum, or attention readout. Controlled readout marks a substation node as controlled whenever at least one of its busbars is controlled. The sparse graph transformer assigns the attachments a dedicated learned relation.

This augmentation differs from the same-substation edges in Section 3.3.1: it introduces a learned hub u_s instead of directly connecting pairs of busbars. It can be combined with the virtual-node readout.

3.4 Pooling and Graph Readout

The final graph embedding is obtained either by pooling node embeddings directly or by reading out a learned virtual node. The output projection and action head are the same in both cases.

3.4.1 Mean, Max, Sum pooling

Mean pooling averages the final node embeddings over the valid nodes in the local graph:

$$\overline{\mathbf{h}}_G = \frac{\sum_v m_v \mathbf{h}_v^{(K)}}{\max(1, \sum_v m_v)}, \tag{3.25}$$

where m_v is the valid-node mask. This is one mean over all valid node types, not one mean per type to be invariant. Thus the busbar graph pools busbar nodes, the disaggregated graph pools busbars, generators, and loads, and the disaggregated-line graph additionally pools transmission-line nodes. One-hop contextual nodes are included in the ordinary mean readout.

No sum pooling because ultimately the objective would be to have something tranfarable to other graph and sum pooling would be dependent on the number of nodes in the graph.

Maximum pooling instead takes an independent maximum in every feature channel:

$$\left(\overline{\mathbf{h}}_G^{\max}\right)_c = \max_{v\,:\,m_v=1} \left(\mathbf{h}_v^{(K)}\right)_c. \tag{3.26}$$

The three readouts differ in what they preserve. The mean is invariant to the number of pooled nodes, but it dilutes an isolated extreme node as the graph grows: one overloaded line among forty nodes

moves the mean by a fortieth of itsown deviation. The sum preserves that node's full contribution and additionally encodes graph size, which the mean discards. The maximum is again invariant to the count and reports the strongest activation per channel, which suits detecting whether some node is in a critical state, at the cost of discarding how many nodes are in that state and of propagating gradients only to the arg-maximum node of each channel.

3.4.2 Virtual-node graph readout

A virtual-node readout replaces terminal mean, sum, maximum, or attention pooling with one learned graph-summary node. It can be combined with any of the three graph schemas.

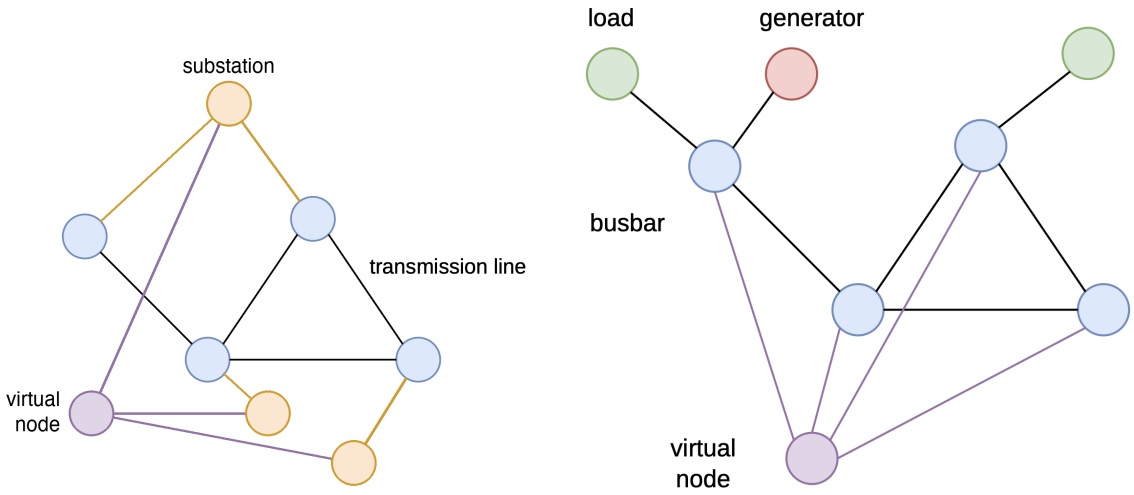
Augmented graph

Let $v_\star$ denote the virtual node. Define the summary set $\mathcal{V}_{\text{sum}}$ as the included substation nodes when the augmentation from Section 3.3.2 is enabled, and as the included busbar nodes otherwise. The augmented graph is

$$\mathcal{V}^+ = \mathcal{V} \cup \{v_\star\}, \tag{3.27}$$

$$\mathcal{E}^+ = \mathcal{E} \cup \{(i, v_\star) \mid i \in \mathcal{V}_{\text{sum}}\}. \tag{3.28}$$

Equation 3.28 shows the `toward_summary` case; `bidirectional` additionally inserts $(v_\star, i)$ for every summary node. Thus the virtual node connects to all included busbars, or to all included substation-summary nodes when that augmentation is enabled. It has no direct edge to generator, load, or transmission-line nodes.



(a) With substation-summary nodes.

(b) Without substation-summary nodes.

Figure 3.6: Virtual-node graph readout, shown for both definitions of the summary set $\mathcal{V}_{\text{sum}}$ in Equation 3.28. The purple node aggregates that set and its final embedding replaces terminal node pooling; reverse messages are optional. In (a) the substation-summary augmentation of Section 3.3.2 is enabled, so the virtual node attaches to the orange substation nodes and reaches the busbars only through them, which gives a two-level hierarchy busbar → substation → virtual node. In (b) the augmentation is disabled and the virtual node attaches directly to every included busbar. In neither case does it connect to generator, load, or transmission-line nodes.

Learned summary instead of terminal pooling

The initial virtual representation $\mathbf{h}_\star^{(0)}$ is learned. A message-passing layer jointly updates the physical and virtual nodes:

$$\left(\{\mathbf{h}_i^{(k+1)}\}_{i\in\mathcal{V}}, \mathbf{h}_\star^{(k+1)}\right) = \text{GNN}^{(k)}\left(\{\mathbf{h}_i^{(k)}\}_{i\in\mathcal{V}}, \mathbf{h}_\star^{(k)}, \mathcal{E}^+\right). \tag{3.29}$$

With `bidirectional` edges the virtual state can also influence its neighbors in later layers; with `toward_summary` it only aggregates. After K layers, the graph embedding is

$$\mathbf{z}_G = \mathrm{ReLU}\left(\mathbf{W}_o \mathbf{h}_\star^{(K)} + \mathbf{b}_o\right), \tag{3.30}$$

rather than from $\mathrm{Pool}(\{\mathbf{h}_i^{(K)}\}_{i\in\mathcal{V}})$. The output dimension is unchanged.

Relation attributes and encoder compatibility

Virtual connections are structural: their continuous edge attributes are zero, except for a neutral unit value in the `rho` position when that feature exists. The sparse graph transformer allocates dedicated learned relations for substation attachments and virtual-node attachments, allowing its attention layers to distinguish hierarchy links from all physical and equipment relations.

Without substation nodes, a graph containing N_{bus} included busbars receives one virtual node and either N_{bus} or $2N_{\text{bus}}$ directed edges. With the hierarchy enabled, the virtual-node stage instead adds either S or $2S$ edges on top of the substation augmentation. After one layer the virtual node has aggregated its immediate busbar or substation neighbors.

3.5 Candidate-Action Scoring from the Explicit-Line Graph

The default action head produces one output column per discrete action. That column is tied to an action index and knows nothing about the physical objects the action modifies. This section describes an alternative head that scores each action from the graph nodes it touches. It is available only for the explicit-line schema of Section 3.2.4 and is selected with `actor_action_head=candidate_pool`; the default remains the action-index head of Appendix A.5. The PPO objective, the categorical policy, sampling, and deterministic evaluation are unchanged.

3.5.1 From action indices to candidate scoring

Throughout this section, $\mathbf{z}_a$ denotes the graph embedding of agent a: the node states of its local graph pooled as in Section 3.4 and passed through the output projection (Equation A.10). It is one 128-dimensional vector per agent per step, and it is the only thing the actor has ever seen of the graph.

With the default head, agent a produces its logit vector in one step from $\mathbf{z}_a$:

$$\boldsymbol{\ell}_a = \mathrm{MLP}_a(\mathbf{z}_a), \qquad \boldsymbol{\ell}_a \in \mathbb{R}^{|\mathcal{A}_a|}. \tag{3.31}$$

The last layer has one row per action index, so what makes an action good must be learned separately for every index, and the head grows with $|\mathcal{A}_a|$.

Candidate scoring replaces that by one shared scalar-valued network applied to every action in turn:

$$\ell_{a,j} = \mathrm{score}\left(\underbrace{\mathbf{z}_a}_{\text{whole graph}}, \underbrace{\mathbf{c}_j}_{\text{nodes action } j \text{ touches}}, \underbrace{\boldsymbol{\phi}_j}_{\text{what action } j \text{ does}}\right). \tag{3.32}$$

The same parameters score all actions, so the head no longer grows with $|\mathcal{A}_a|$, and two actions are ranked by the difference between their touched-node contexts rather than by two independent output rows. This is what makes the explicit line nodes usable: an action next to a loaded line is scored from that line's own embedding.

Two ingredients are needed: a static table saying which nodes each action touches (Section 3.5.2), and a rule for turning those nodes into the context vector $\mathbf{c}_j$ (Section 3.5.3).

3.5.2 What each action touches

The first ingredient is one lookup table per agent, with one row per action, answering a single question: which nodes of the local graph does this action physically touch? It is built once, when the environment is created, and never recomputed during training.

A Grid2Op topology action states which entries of the topology vector it changes, and each entry belongs to a known object: a line endpoint, a generator, or a load. Reading those entries gives the objects the action modifies, and the graph builder's row maps turn each object into the row of the graph that holds its embedding. Table 3.9 follows one action through this.

Table 3.9: One action, from Grid2Op to node rows: moving the origin end of line 4 onto the other busbar of substation 6. Row indices are illustrative.

Step	Result for this action
Objects the action changes	Substation 6, line 4
Their rows in the graph	Both busbars of substation 6, here 12 and 13, and the line node of line 4, here 22
Stored for scoring	Busbar [12, 13], line [22], generator and load empty

Two decoding rules are worth stating. A touched substation contributes *all* of its busbar rows, not only the busbar in use, because the action changes which of them the equipment sits on. An action that connects or disconnects a line contributes that line and *both* of its endpoint substations, because the connectivity changes at each end. The rows are stored as fixed-size arrays with a mask, padded to the widest action, so that a single gather serves the whole action space.

Write $\mathcal{N}_{\text{bus}}(j)$, $\mathcal{N}_{\text{line}}(j)$, $\mathcal{N}_{\text{load}}(j)$, and $\mathcal{N}_{\text{gen}}(j)$ for the four sets of node rows that action j touches. Because a topology change moves line endpoints whose line node may sit at the edge of the local graph, the head requires one-hop neighbor context; the builder raises an error rather than pool an incomplete context.

Each action also carries a static feature vector $\boldsymbol{\phi}_j \in \mathbb{R}^{12}$, listed in Table 3.10. It describes what the action does independently of the current grid state, so the scorer can tell a single endpoint move from an action that reconfigures a whole substation even when both touch the same nodes.

Table 3.10: The static action-feature vector $\boldsymbol{\phi}_j$. The five indicators are binary; the seven counts are divided by their largest value over that agent's action space, so every entry lies in $[0, 1]$ and is independent of the grid size.

Entry	Meaning
Indicators	
`is_do_nothing`	The action is the idle action
`has_busbar_context`	It modifies at least one substation
`has_line`	It touches at least one line
`has_load`	It touches at least one load
`has_generator`	It touches at least one generator
Scaled counts	
`n_substations`	Substations modified
`n_topology_changes`	Topology-vector entries changed
`n_line_status_changes`	Lines connected or disconnected
`n_line_endpoint_changes`	Line endpoints moved to another busbar
`n_generator_changes`	Generators moved to another busbar
`n_load_changes`	Loads moved to another busbar
`n_other_changes`	Changed objects of any other type

3.5.3 Pooling the touched nodes

The second ingredient turns the touched rows into $\mathbf{c}_j$. In $\mathbf{z}_a$ the nodes have already been averaged away, so the encoder also returns the node states themselves:

$$(\mathbf{z}_a, \mathbf{H}) = \text{encoder}_{\text{nodes}}(\mathcal{G}_t^{(a)}), \qquad \mathbf{H} \in \mathbb{R}^{|\mathcal{V}| \times d_h}, \tag{3.33}$$

where row v of $\mathbf{H}$ is the state $\mathbf{h}_v^{(K)}$ of node v after the last message-passing layer. These are exactly the vectors that mean pooling averages into $\mathbf{z}_a$; nothing new is computed, and a run with the default head is unaffected. Only the physical nodes appear in $\mathbf{H}$: substation-summary and virtual nodes are excluded, since an action touches equipment and not a learned hub.

The basic operation is a masked mean over the rows of one object type,

$$\mathbf{c}_j^{(t)} = \frac{1}{\max\left(1, |\mathcal{N}_t(j)|\right)} \sum_{v \in \mathcal{N}_t(j)} \mathbf{h}_v^{(K)}, \tag{3.34}$$

where the clamped denominator makes an untouched type give the zero vector instead of a division by zero. Table 3.11 lists the ways of combining these.

Table 3.11: Pooling modes. Only the first two are implemented; `typed_mean` is the default.

Mode	Context $\mathbf{c}_j$	Width	What it buys
`mean`	One masked mean over all touched rows, whatever their type	d_h	The smaller control; mixes roles together
`typed_mean`	The four typed means of Equation 3.34, concatenated	$4d_h$	The scorer can tell a stressed line from the load beside it
`attention`	The action queries its own touched rows and weights them	d_h	Could focus on the one endpoint that matters; not implemented

Attention pooling is left for after the typed-mean baseline has been measured, because it adds parameters and a second failure mode to a head whose value is not yet established.

3.5.4 Scoring, including the idle action

All non-idle actions are scored by one network with a single output unit:

$$\ell_{a,j} = \text{MLP}_{\text{score}}\left(\left[\mathbf{z}_a; \mathbf{c}_j; \boldsymbol{\phi}_j\right]\right), \qquad j \geq 1. \tag{3.35}$$

The idle action touches nothing, so its pooled context is identically zero and would otherwise be scored by a network trained on non-zero contexts. It therefore gets its own head over the graph embedding alone:

$$\ell_{a,0} = \text{MLP}_{\varnothing}(\mathbf{z}_a) + b_{\varnothing}, \tag{3.36}$$

with $b_{\varnothing}$ a learned scalar applied to action 0 alone. This makes the idle logit an explicit judgment of whether the local grid state is safe enough not to intervene.

3.5.5 Cost and current limits

The candidate contexts form a tensor of shape $[\text{batch}, |\mathcal{A}_a|, d_c]$, so memory grows with the size of the action space. This is the reason to start on `bus14` and on reduced action spaces before the full WCCI one.

Two properties matter when comparing this head against the default. The head no longer scales with $|\mathcal{A}_a|$, so the two actors do not have the same parameter count even at identical graph settings, and both counts should be reported. The idle logit also comes from a different mechanism than the other logits, so the action-0 frequency has to be compared against the action-index baseline rather than assumed comparable.

The head currently requires the explicit-line graph with one-hop neighbor context, does not yet combine with the intervention gate, and implements only the first two pooling modes of Table 3.11. Auxiliary heads over the same node embeddings, such as predicting which lines are about to overload, are a possible later addition.

3.6 Graph Actor and Critic Configuration

The actor uses two message-passing layers with 128 hidden features. LayerNorm is applied after the input node projection and after each message-passing layer. The graph readout produces a 128-dimensional embedding, followed by an agent-specific action head with two 128-unit hidden layers. The graph encoder and output projection are shared across agents; the action heads are separate because their action-set sizes may differ. This is the default `actor_action_head=mlp` setting; the candidate-scoring alternative of Section 3.5 replaces only this head. The flat observation is not concatenated to the graph embedding, and learned global node-identifier embeddings are disabled. Local actors include one-hop neighboring context.

The centralized critic is a separate MLP over the normalized flat state and does not reuse the actor graph embedding. It is used only during centralized training. Graph-input normalization follows Section 3.1 and is independent of the critic's flat-state normalization. The exact GINE and GAT updates, graph readout, output projection, and action-head equations are given in Appendix A.

3.7 Action-Space Reduction for Large Grids

The representations above change what an agent sees. This section changes what it can do, and is what makes the larger grids tractable at all: the WCCI bus36 setting has a far larger discrete topology-action space than `bus14`, so before training MAPPO on it the action space is reduced offline from simulated action outcomes. During rollouts, states are collected when the grid is stressed,

$$\rho_{\max} > 0.90.$$

For each collected state s, candidate actions are simulated and compared against do nothing:

$$\Delta(s, a) = \rho_{\text{after}}(s, a) - \rho_{\text{after}}(s, 0).$$

An action is counted as useful when it lowers the post-action loading relative to do nothing, i.e. $\Delta(s, a) < -\epsilon$. Each action is then scored by its empirical improvement rate,

$$\text{score}(a) = \frac{\#\{\text{tested states where } a \text{ improves the grid}\}}{\#\{\text{tested states where } a \text{ is evaluated}\}},$$

with a minimum-count filter to avoid selecting rarely sampled actions. The final reduced action space keeps the best actions per agent, while always preserving action 0. The candidate sets are then sanity-checked with a one-step simulator-greedy controller: at each risky state, the controller simulates the available reduced actions and executes the best improving one. That check also sizes the reduced space, since it shows how much survival is still reachable once the action set is cut. Table 3.12 reports it for five sizes; the survival of trained agents on this environment is a separate question and is not covered here.

Table 3.12: One-step greedy evaluation of WCCI reduced action spaces on 50 random test chronics. All rows use the same chronic sample (`chronic_sample_seed=0`); the do-nothing mean survival on this sample is 28.30%.

Reduced action space	Kept actions per agent (a_0, a_1, a_2, a_3)	Greedy survival (%)	Do-nothing survival (%)	Survival delta (%)	Greedy full survival (%)
Do-nothing reference	$1, 1, 1, 1$	–	28.30	–	–
WCCI $h = 1$, `mk64`	$64, 64, 64, 64$	55.24	28.30	26.94	22.00
WCCI $h = 1$, `mk128`	$77, 128, 127, 128$	60.65	28.30	32.35	22.00
WCCI $h = 1$, `mk256`	$77, 256, 127, 256$	68.61	28.30	40.31	36.00
WCCI $h = 1$, `mk512`	$77, 512, 127, 512$	71.21	28.30	42.90	40.00
WCCI $h = 1$, `mk1024`	$77, 1024, 127, 1024$	78.18	28.30	49.87	54.00

Chapter 4

Graph-Design Screening

This chapter reports the graph-design screens. All are balanced full factorials on `bus14` with three seeds per cell, and all ask the same kind of question: does a specific choice in the actor of Chapter 3 change how well a decentralized MAPPO policy survives on held-out chronics?

- **Screen A** varies the depth and width of the actor encoder, which applies to every graph schema: a 3×4 grid, 36 runs. *Seed 0 complete, seeds 1 and 2 training.*
- **Screen B** varies the message-passing operator and the readout, GINE against GAT and mean against maximum pooling: a 2×2 factorial, 9 new runs plus one reused cell. *Training at the time of writing.*
- **Screen C** varies the three structural augmentations of the busbar graph – same-substation edges, substation-summary nodes, and virtual-node readout (Sections 3.3 and 3.4.2): a 2^3 factorial, 24 runs. *Complete.*
- **Screen D** varies the direction of the generator and load attachment relations in the disaggregated graph (Section 3.2.3): a 3×3 factorial, 27 runs. *Complete.*

Screens A, B, and C all run on the busbar graph and share one reference configuration, so they can be read as a sequence; only Screen D changes the schema. Screens C and D are complete, 51 finished runs between them; Screen A has its first seed.

4.1 How This Chapter Is Organized

Each screen is presented as one block containing its design and its outcome together, a compressed form of the reporting pattern of Section 2.1:

1. **Question.** Which design choice is uncertain?
2. **Design.** Which factors are varied, at which levels, and what is held fixed.
3. **Result.** The measured tables.
4. **Reading.** What follows, and what does not.

4.2 Shared Protocol

Every screen uses the configuration in Table 4.1. Every setting not named as a screened factor is identical across all runs, so each screen is a paired comparison against its own baseline cell.

Table 4.1: Settings shared by all screens. Only the graph type and the screened factors differ between them.

Component	Fixed setting
Environment	`bus14`, difficulty 0, topology actions, decentralized agents, normalized observations and rewards, 80/20 train/test chronic split with split seed 0
Actor	One GNN encoder shared by all agents; LayerNorm; node pre-encoder; two 128-unit action-head layers; one-hop neighbor context
Critic	Centralized MLP with three 256-unit hidden layers
Graph preprocessing	None: neither deterministic physical scaling nor running standardization
Training budget	15,000,000 environment steps, 72 parallel environments, 576 rollout steps
MAPPO update	10 epochs, 16 minibatches, actor and critic learning rates 10^{-4} with linear annealing, $\gamma = 0.9$, $\lambda = 0.95$, clip 0.2, target KL 0.02, entropy coefficient 0.01
Exploration	No initial do-nothing prior and no action-0 logit bonus
Sparse control	Disabled: no intervention gate, no intervention penalty, no evaluation-time rho heuristic
Evaluation	Deterministic evaluation every 82,944 environment steps, 10 episodes on each split
Seeds	0, 1, 2

4.2.1 Endpoint statistics

Survival and the train/test split are as defined in Section 2.2: survival is the percentage of an episode's maximum length that the agent keeps the grid alive, reported on the test split. What differs here is how a run is reduced to one number. Each run saves the checkpoint with the highest test survival seen during training. That checkpoint is then re-evaluated deterministically over all 201 held-out test chronics.

Every run reaches the 15M-step budget, to within a negligible margin, so the screens do not report how far their runs got. Each cell of a screen aggregates its seeds over the same 201 chronics.

4.2.2 Compute

The runs were executed on two clusters, JED (CPU) and IZAR (GPU); seeds of the same cell are spread across both, so the compute backend is not confounded with any screened factor.

Training is deterministic at a fixed seed. Two separate launches of one configuration produced the same best checkpoint, at the same step, with the same full-test survival to full precision. Run-to-run variation at a fixed seed is therefore zero, and every difference reported in this chapter comes from the configuration or from the seed.

What limits the campaign is simulation throughput rather than model size. A training step is dominated by stepping 72 Grid2Op environments, so the actor forward and backward passes are a small part of the wall clock: across the depth and width grid of Section 4.3 the actor varies by a factor of three in parameters, 83.6k to 244.9k, without a comparable effect on run time. Parameter count is therefore not the cost to minimize when choosing an architecture here.

4.3 Screen A: Encoder Depth and Width

Question

We need to fix a message-passing depth K and an encoder width d_h. They apply to every schema, and every other screen in this chapter holds them constant while varying something else, so the pair has to be defensible before any of those comparisons mean anything. This screen asks which pair we should standardize on.

The two factors do different work on the busbar graph, which is the only schema this grid uses. Depth is a question of reach: with K layers a node embedding sees K hops away, and here one hop already crosses a transmission line into an adjacent substation. A local actor graph extends only one substation beyond the controlled ones, so by two layers a controlled busbar has already reached the edge of its own graph and a third layer mostly re-mixes what the second collected. Width is a question of capacity rather than reach: it sizes every node embedding and the pooled vector the action head consumes, and it is what limits how much of the local state survives pooling.

Because the grid runs on the busbar graph alone, what it measures is depth and width for that schema; carrying the chosen pair over to a schema with explicit asset or line nodes is an assumption rather than a measurement.

Design

A complete 3×4 grid with three seeds per cell, 36 runs. Table 4.2 lists the screened parameters and their levels. The graph output dimension is matched to the hidden dimension, so a cell varies one width rather than two. Every other setting is inherited from the reference configuration of Section 4.2, seed by seed.

Table 4.2: Screen A. Screened parameters and the values tested. All other settings are held at Table 4.1.

Parameter	Setting	Values tested
Message-passing layers K	`gnn_layers`	1, 2, 3
Encoder width d_h	`gnn_hidden_dim`	16, 32, 64, 128

Twelve of these runs have finished so far: seed 0 of each of the twelve architectures. Everything below therefore rests on one run per cell, so a difference between two cells cannot yet be separated from the variation another seed would have produced.

Result

Table 4.3: Screen A. Full-test survival of the best checkpoint, one seed per cell, by message-passing depth and encoder width. The reference depth and width used by every other screen is `mp2_h128`.

	Encoder width d_h			
Layers K	16	32	64	128
1	**98.68**	97.07	96.54	84.18
2	84.19	67.39	68.38	96.66
3	20.50	93.39	92.97	100.00[†]

[†]Best checkpoint selected at 0.17M steps; see the reading.

Table 4.4: Screen A. Marginal effect of each factor, averaged over the other. Four runs per depth, three per width.

Level	Mean (%)	Min (%)	Max (%)
Message-passing depth			
$K = 1$	94.12	84.18	98.68
$K = 2$	79.15	67.39	96.66
$K = 3$	76.71	20.50	100.00
Encoder width			
$d_h = 16$	67.79	20.50	98.68
$d_h = 32$	85.95	67.39	97.07
$d_h = 64$	85.96	68.38	96.54
$d_h = 128$	93.61	84.18	100.00

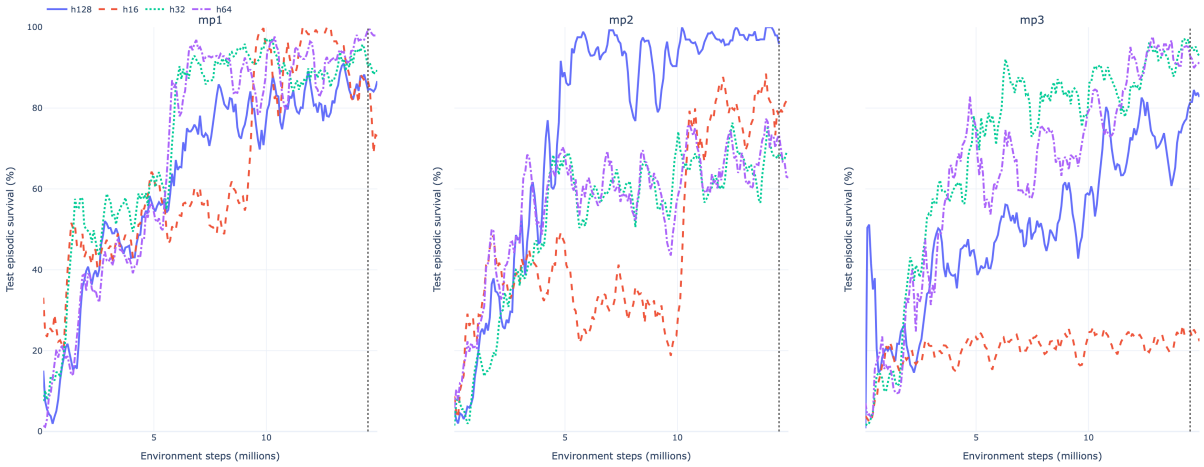


Figure 4.1: Screen A. Test episodic survival during training, one panel per message-passing depth, with the four widths compared inside each panel.

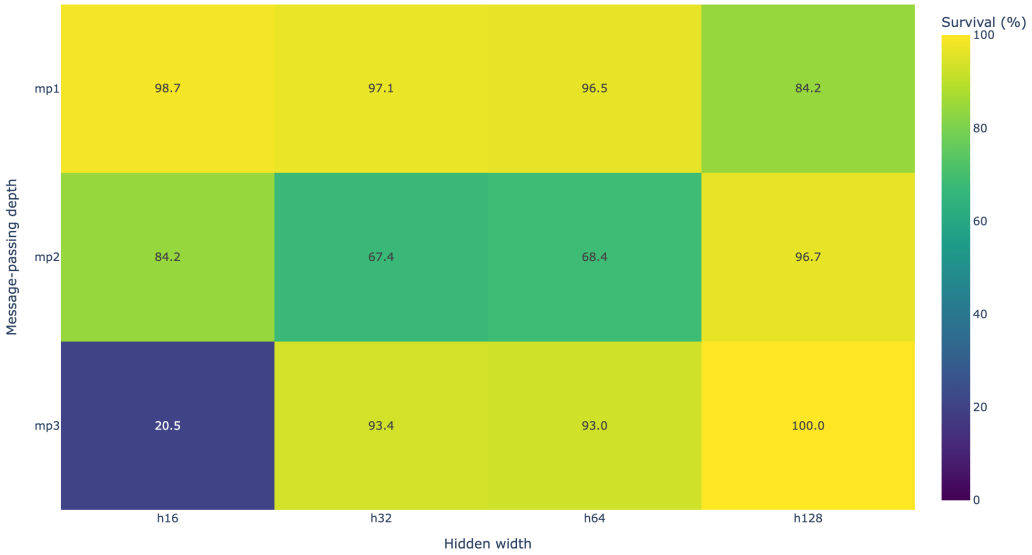


Figure 4.2: Screen A. The same twelve cells as a heatmap of the full-test survival of each run's best checkpoint.

Figure 4.3: Screen A. Full-test survival against actor parameter count on a logarithmic axis, joined within each depth. The grid spans 83.6k to 244.9k actor parameters, a factor of three.

Reading

One cell is unexplained. `mp3_h128` survives all 201 test chronics, the only perfect score measured anywhere, from a checkpoint saved after 0.17M steps. The surrounding facts are unusual rather than obviously wrong: the 10-episode evaluation at step 165,888 returned exactly 1.000, that checkpoint was kept as the run's best, and no later evaluation matched it, the same run ending training between 0.63 and 0.85. The score itself is a deterministic evaluation over the complete held-out split, so it is not a small-sample accident of the kind a 10-episode evaluation is prone to: a policy trained for 0.17M steps did survive every test chronic. Why an early checkpoint should generalize better than anything the same run reached later is not something this screen can answer, and it is worth returning to. Until it is explained, the cell is reported but is not used to argue for depth or width.

At the opposite extreme, `mp3_h16` reaches 20.50%, within a quarter of a point of a policy that never acts. That run did not learn, so its cell measures inaction rather than the effect of depth.

Depth does not separate cleanly on the endpoint statistic. The $K = 1$ row averages 94.12%, above $K = 2$ at 79.15% and $K = 3$ at 76.71%, but the three rows overlap heavily: the best and the worst cell of the whole grid, `mp3_h128` and `mp3_h16`, are both at $K = 3$, and the $K = 1$ row spans 84.18% to 98.68%. A row mean built from four single-seed cells with that much internal range does not establish an ordering, so the depth marginal is an observation here and not a result.

Width behaves more simply: wider is safer. Averaged over depth, survival rises at every step in width, 67.8%, 86.0%, 86.0%, 93.6%. The clearer way to read it is the worst case rather than the average. The weakest cell at $d_h = 16$ survives 20.5%, at $d_h = 32$ it is 67.4%, and at $d_h = 128$ it is 84.2%. Wide encoders are not dramatically better when a run goes well; they are much better when it goes badly, and the narrowest ones sometimes fail to learn at all.

The endpoint statistic is strong evidence, but not the only evidence. Each value is a deterministic evaluation over all 201 test chronics, which makes it far more reliable than any single training-time evaluation. What it does not account for is which checkpoint of a run is evaluated and which seed the run was trained from. Both move the number: the checkpoint is picked by a maximum over roughly 180 ten-episode evaluations, and with one seed per cell there is no second draw to average against. The training curves address the first of the two, since they use every evaluation of a run rather than the one that happened to be highest. Table 4.5 summarizes them three ways.

Table 4.5: Screen A. Convergence measured from the training curves rather than from a single checkpoint. Steps to 90% is the first evaluation at which the smoothed test survival reaches 90%; the curve mean is the time-average of the smoothed curve over training; the final-window mean averages the last ten evaluations. Dashes mark architectures that never reached the threshold.

Architecture	Steps to 90% (M)	Curve mean (%)	Final-window mean (%)
`mp2_h128`	**4.81**	**75.1**	**98.5**
`mp1_h32`	6.14	73.3	91.5
`mp1_h64`	6.55	69.9	98.0
`mp3_h32`	6.30	69.8	95.3
`mp1_h16`	9.46	64.6	81.3
`mp3_h64`	11.78	63.9	92.8
`mp1_h128`	13.35	63.2	86.0
`mp2_h64`	–	56.5	69.1
`mp2_h32`	–	52.6	68.3
`mp3_h128`	–	50.9	80.7
`mp2_h16`	–	46.7	79.4
`mp3_h16`	–	20.4	24.3

`mp2_h128` leads all three: it is the only architecture to reach 90% before 5M steps, it has the highest curve mean, and it ends training highest. Its one near-tie is the 80% threshold, which `mp3_h64` crosses 0.08M steps earlier. The reordering against Table 4.3 is itself informative: `mp1_h16` is first on the best-checkpoint statistic and sixth here, because its curve reaches 100% once and averages 81.3% at the end, while `mp3_h128` is first there and tenth here.

Which pair to standardize on. Three considerations point the same way. On the endpoint statistic the cells between 92% and 99% are close enough that one seed cannot order them, so that table alone does not select a winner. On the curves, which use every evaluation rather than the highest one, `mp2_h128` is the clearest and fastest learner of the twelve.

The third consideration is the grid itself. Two message-passing layers are what the `bus14` local graphs ask for: inside each agent's observation every pair of controlled substations lies within two line hops, with three exceptions in the largest region, that of agent 2, where three pairs are three hops apart. With $K = 2$ a controlled busbar can therefore integrate almost all of the region it acts on, whereas $K = 1$ leaves part of that region out of reach and $K = 3$ mostly re-mixes what the second layer already gathered. Depth is matched to the size of the observation rather than chosen for capacity, and since simulation throughput rather than parameter count limits the campaign (Section 4.2.2), the extra parameters of the wider encoder cost little.

The evidence therefore supports $(K, d_h) = (2, 128)$: it is the baseline for the screens in this chapter and the depth and width used for the rest of the study.

What this does not license. Choosing a default is one question; quantifying how much depth and width matter is another, and one seed per cell cannot answer the second. Wider encoders are visibly the safer ones and the $K = 1$ row is the most consistent on the endpoint statistic, but neither observation survives a single seed on its own, so neither is reported here as an effect. Seeds 1 and 2 would turn them into one, and would also show whether the perfect score of `mp3_h128` reappears or was particular to that run.

4.4 Screen B: Encoder Operator and Readout

Runs in progress. The design below is final; the coverage, result, and reading subsections are placeholders to be filled from the completed runs.

Question

Two independent choices have been held fixed at their defaults. The message-passing operator can be GINE, which adds the projected edge vector to the sender state inside every message, or GAT, which uses the edge vector to weight how strongly each incoming message counts (Appendix A). The readout can be the mean over valid nodes or a per-channel maximum (Section 3.4.1). The two interact in principle: a softmax-weighted aggregation followed by a maximum readout selects twice, while mean pooling after an unweighted sum preserves how many nodes are in a given state. Which combination suits the busbar graph?

The busbar schema is the relevant setting for a second reason. Its edge vector carries only the physical line block and no relation one-hot (Table A.3), so GAT's attention here is computed from line state — loading, status, cooldown — rather than from relation identity. If attention helps anywhere, a graph whose edges differ only by physical measurement is where it should show.

Design

A 2×2 factorial over encoder $\in \{\text{GINE}, \text{GAT}\}$ and readout $\in \{\text{mean}, \text{max}\}$ with three seeds per cell. Only three cells are newly launched — 9 runs in `configs/gnn_graph_screening/stage3_encoder_pooling` — because the GINE/mean cell is the configuration of Table 4.1 at the depth and width fixed in Section 4.3, and is reused rather than repeated. GAT uses four heads averaged rather than concatenated, so all four cells return the same 128 features. Every other setting is inherited from the reference configuration seed by seed.

Reusing a cell would be a problem if a re-launch could disagree with the original, but training is deterministic at a fixed seed (Section 4.2.2), so the inherited numbers are exactly what a re-run would produce. Any difference against the three new cells is therefore a configuration difference and not launch noise.

Result

Table 4.6: Screen B. Test survival by encoder and readout, averaged over three seeds, with the seed-to-seed standard deviation in parentheses. The GINE/mean cell is inherited from the reference configuration. *[pending: fill the three new cells using the same endpoint statistic as the inherited cell.]*

	Readout	
Encoder	Mean	Max
GINE	97.31 (1.22)[†]	–
GAT	–	–

[†]Reference configuration, full-test survival of the best checkpoint; not a new run.

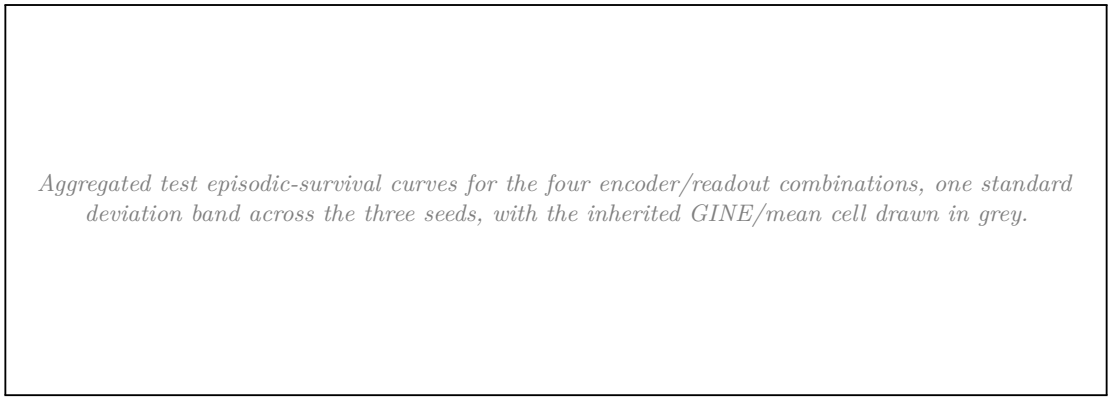


Figure 4.4: Screen B. Survival curves by encoder and readout. *[pending: replace the box with the exported figure.]*

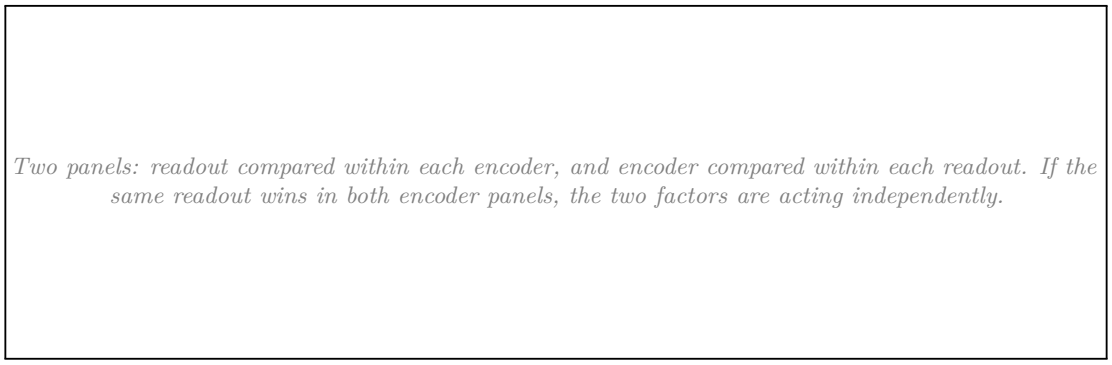


Figure 4.5: Screen B. Factors compared within each other. *[pending: replace the box with the exported figure.]*

Reading

[pending: report the four cells, then answer three questions.] Whether either factor has a main effect large relative to the seed spread; whether they interact, which the 2×2 can show but not explain; and whether any gain from GAT is worth its cost per step, since a policy that reaches the same survival with a cheaper operator is the better default for the larger grids. State explicitly whether the inherited GINE/mean cell was re-run for confirmation.

4.5 Screen C: Structural Augmentations of the Busbar Graph

Question

Three optional augmentations add computational structure to the busbar graph without adding any physical measurement: same-substation edges (e), which let the busbars of one substation exchange messages directly; substation-summary nodes (n), which insert one learned hub per substation; and virtual-node readout (v), which replaces mean pooling with a learned graph-summary node. Each is motivated by the same intuition, shorten the path between related busbars, so the question is whether any of them earns its cost, and whether they compose.

Design

A complete 2^3 factorial with three seeds per cell, 24 runs. The graph type is `bus` throughout and the baseline cell is the unaugmented `e0n0v0`, which is the reference configuration derived from Section 4.3

and Section 4.4. Cells are named by their three switches, so `e1n0v1` means same-substation edges on, summary nodes off, virtual-node readout on.

Result

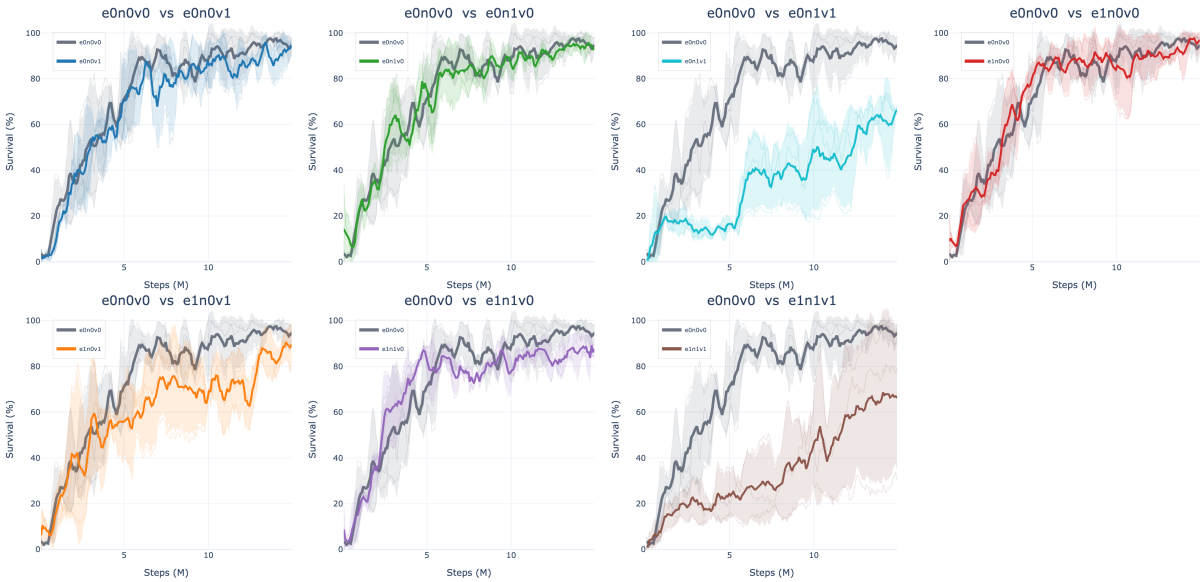


Figure 4.6: Screen C. Test episodic survival during training, one panel per augmented cell against the unaugmented `e0n0v0` baseline in grey. Thin lines are the three seeds, the thick line is their mean, and the band is one standard deviation.

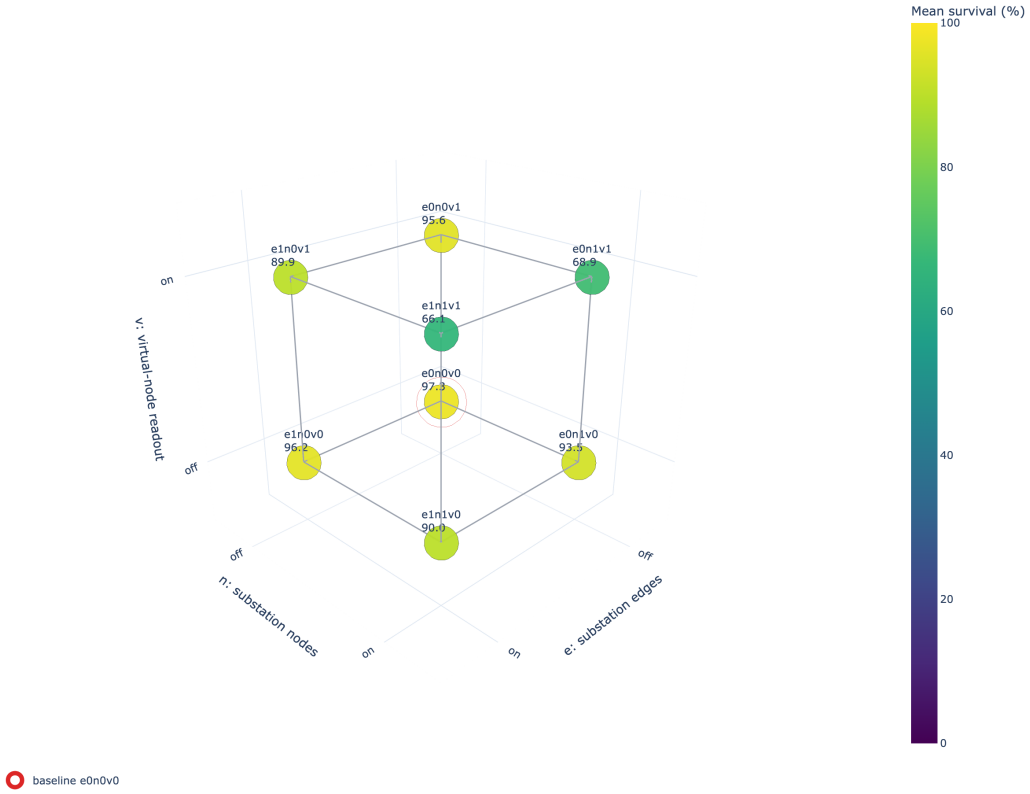


Figure 4.7: Screen C. The factorial as a cube: each vertex is one of the eight cells and each edge flips a single factor with the other two held fixed. The baseline `e0n0v0` is circled in red.

Figure 4.8: Screen C. The same eight cells as a heatmap of the full-test survival of each run's best checkpoint, with the three individual seed values printed under each cell mean.

Table 4.7: Screen C. Cells ranked by full-test survival, with the difference from the unaugmented `e0n0v0` baseline.

Cell	Mean (%)	Std (pp)	Min (%)	Max (%)	Δ baseline (pp)
`e0n0v0`	97.31	1.22	96.55	98.72	0.00
`e1n0v0`	96.20	3.94	91.66	98.68	−1.11
`e0n0v1`	95.63	6.19	88.52	99.82	−1.68
`e0n1v0`	93.51	6.39	86.72	99.40	−3.80
`e1n1v0`	90.01	4.92	85.42	95.19	−7.30
`e1n0v1`	89.92	9.42	80.48	99.33	−7.39
`e0n1v1`	68.91	5.77	62.59	73.91	−28.40
`e1n1v1`	66.11	33.95	27.86	92.68	−31.20

Table 4.8: Screen C. Main effect of each factor (12 runs per level) and the four single-factor flips of the cube, each one enabling that factor with the other two held fixed. `Spread` is the range of those four flips.

Factor	Off (%)	On (%)	Effect (pp)	Flips (pp)	Mean flip (pp)	Spread (pp)
e (substation edges)	88.84	85.56	−3.28	−1.1, −5.7, −3.5, −2.8	−3.28	4.60
v (virtual readout)	94.26	80.14	−14.11	−1.7, −24.6, −6.3, −23.9	−14.11	22.92
n (summary nodes)	94.77	79.63	−15.13	−3.8, −26.7, −6.2, −23.8	−15.13	22.92

Reading

No augmentation earns its place, and the result is unambiguous: the unaugmented `e0n0v0` baseline is the best cell at 97.31%, and all twelve single-factor flips in Table 4.8 are negative. Not one of the three

mechanisms helps in any context. The baseline is also the most stable cell in the screen, with a seed spread of 1.22 points against 3.9–33.9 for every augmented cell, so the augmentations cost reliability as well as survival. The training curves in Figure 4.6 separate the cells well before the budget ends, so this is not an artefact of where the runs were cut.

The two hierarchy factors dominate, at −15.13 points for summary nodes and −14.11 for virtual-node readout, and they are not independent: each is mild in the cell where the other is off (n costs 3.80 points at `e0v0`; v costs 1.68 at `e0n0`) and severe once the other is on, with both $n1v1$ cells landing between 66% and 69%. The upper face of the cube in Figure 4.7 shows this directly, and the spread column of Table 4.8 makes it explicit: for n and v the four flips range over 23 percentage points, so the main effect alone is not a usable summary of either factor. Two of the weaker runs also peaked early and never recovered: the `e0n1v0` seed-1 checkpoint was selected at 2.65M steps and the `e1n1v1` seed-2 checkpoint at 9.87M, against 13.7M to 14.9M for the rest.

The interpretation is that the two hierarchy mechanisms are redundant rather than complementary. Both insert a learned aggregation node above the busbars, and the virtual node attaches to the summary nodes when they exist (Figure 3.6), so enabling both stacks two learned hubs in series between the busbars and the readout. With only two message-passing layers, the readout of the `n1v1` cells is separated from most physical nodes by more hops than the encoder has layers. Screen A varies exactly that depth, so its $K = 3$ row is the direct test of this explanation.

Same-substation edges are the mildest of the three, at −3.28 points on average and within a narrow band: its four flips range only from −1.11 to −5.71, so unlike n and v it behaves like a small, consistent cost rather than a context-dependent one. Its best cell `e1n0v0` sits 1.11 points below the baseline, which is inside the seed spread of both cells, and the per-seed values in Figure 4.8 show why nothing stronger can be claimed: two of its seeds reach 98.7% and 98.3% while the third reaches only 91.7%, straddling the baseline's much tighter 96.6–98.7%. The honest reading is that the edges neither help nor clearly hurt on their own, and that they add candidate edges for no measured return.

The operational conclusion is to keep the plain busbar graph with mean readout for subsequent stages, and to treat any future hierarchy proposal as needing a deeper encoder rather than another augmentation on top of two layers.

4.6 Screen D: Generator and Load Message Direction

Question

In the disaggregated graph, each generator and load is a node attached to its busbar. Section 3.2.3 leaves the direction of that attachment open: the asset may send messages to the busbar, receive them from it, or both. The two directions are not symmetric in effect, because the mean readout of Section 3.4.1 pools asset nodes as well as busbars. Does the choice matter, and do the generator and load relations behave independently?

Design

A complete 3×3 factorial over the generator and load directions with three seeds per cell: 27 runs. The graph type is `heterogeneous` throughout; the line and summary directions stay `bidirectional` and are inert for this schema. The encoder matches the reference configuration: GINE, two layers, 128 features, mean readout, no augmentations, so the only screened factors are the two attachment directions. The baseline cell is bidirectional on both relations. Directions are abbreviated `bi` (bidirectional), `a2b` (asset to busbar), and `b2a` (busbar to asset).

Result

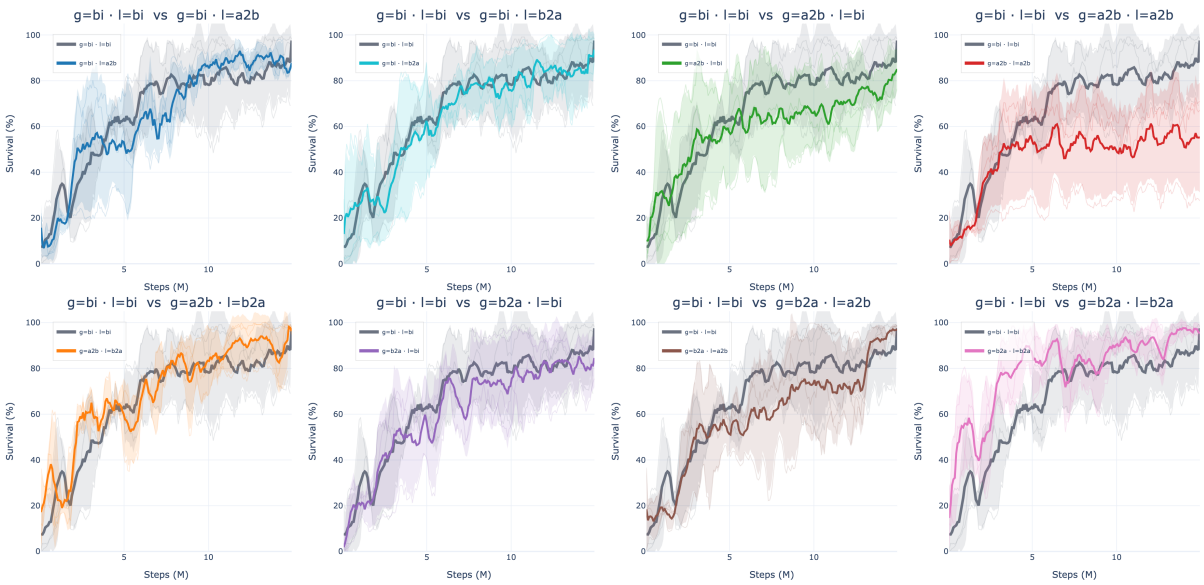


Figure 4.9: Screen D. Test episodic survival during training, one panel per direction pair against the bidirectional baseline in grey. Thin lines are the three seeds, the thick line is their mean, and the band is one standard deviation.

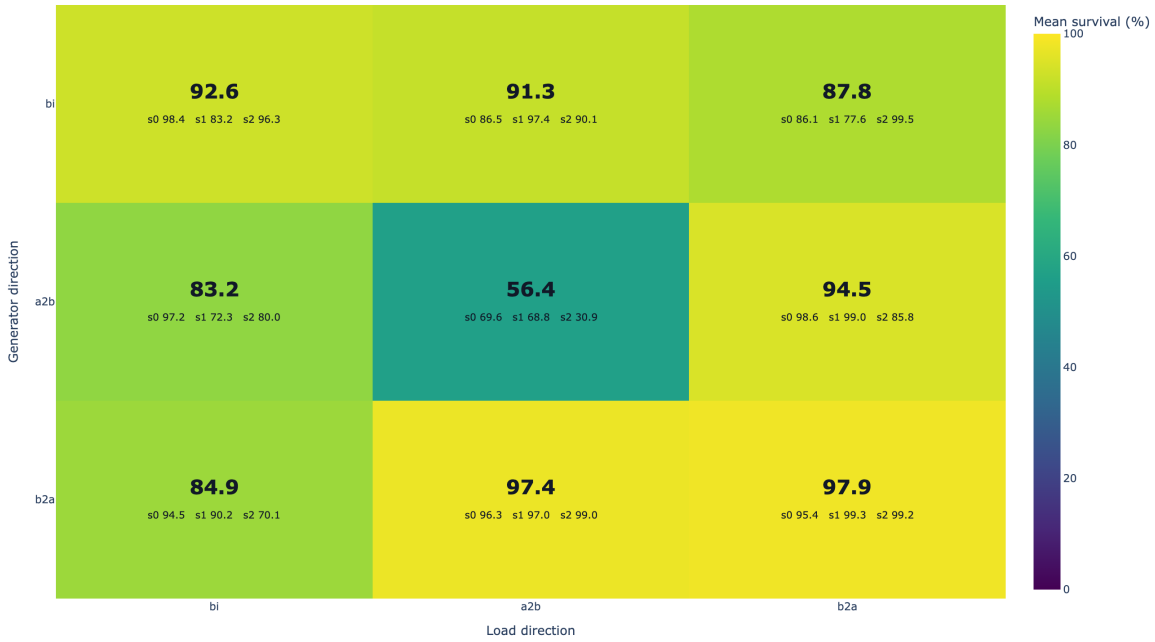


Figure 4.10: Screen D. Full-test survival of the best checkpoint by generator direction (rows) and load direction (columns), with the three individual seed values printed under each cell mean. The baseline cell is `bi/bi`.

Table 4.9: Screen D. Direction pairs ranked by full-test survival, with the difference from the `bi/bi` baseline.

Generator / load	Mean (%)	Std (pp)	Min (%)	Max (%)	Δ baseline (pp)
`b2a` / `b2a`	97.94	2.20	95.40	99.25	+5.30
`b2a` / `a2b`	97.43	1.42	96.32	99.03	+4.79
`a2b` / `b2a`	94.47	7.53	85.78	99.03	+1.83
`bi` / `bi`	92.65	8.24	83.21	98.40	0.00
`bi` / `a2b`	91.35	5.54	86.52	97.40	−1.30
`bi` / `b2a`	87.76	11.04	77.64	99.53	−4.89
`b2a` / `bi`	84.93	13.01	70.11	94.46	−7.72
`a2b` / `bi`	83.16	12.75	72.27	97.19	−9.49
`a2b` / `a2b`	56.41	22.12	30.88	69.60	−36.24

Reading

Busbar-to-asset is the better direction for both relations. Averaging over the other relation, the generator marginal is 93.43% for `b2a`, 90.58% for `bi`, and 78.01% for `a2b`; the load marginal follows the same order at 93.39%, 86.91%, and 81.73%. The strongest part of the result is not the mean but the spread: the two best cells, `b2a`/`b2a` and `b2a`/`a2b`, have seed standard deviations of 2.20 and 1.42 percentage points against 8.24 for the bidirectional baseline, and their worst seeds (95.40% and 96.32%) are above the baseline's mean. The seed values printed in Figure 4.10 show this at a glance: the bottom row holds three tightly grouped seeds per cell, while the baseline cell spans 83.2% to 98.4%.

The failure case is equally clear. Making both relations asset-to-busbar collapses the policy to 56.41% with a 22.12-point spread and a worst seed at 30.88%, which is 36 points below the baseline and the largest single effect measured anywhere in this chapter. Its panel in Figure 4.9 separates from the baseline within the first two million steps and never recovers, so this is a training failure rather than a late collapse; one of its seeds never improved on a checkpoint from 4.31M steps.

A plausible mechanism, consistent with the readout defined in Section 3.4.1 but not directly measured here, is that the mean pooling includes generator and load nodes. Under `b2a` an asset node receives busbar context and its updated embedding enters the readout, so the asset row carries information about its local neighborhood. Under `a2b` the asset node never receives a message, so after K layers its embedding is still a projection of its own raw features, and it dilutes the pooled vector with context-free rows; with both relations set that way, every generator and load row is inert.

Chapter 5

Sparse Intervention Control

Every experiment so far has maximized one quantity, survival on held-out chronics. This chapter changes the objective. A policy that keeps the grid alive by reconfiguring substations at every opportunity is not what a control room wants: interventions have an operational cost, and an agent that acts constantly is neither auditable nor trustworthy. The question here is whether the same decentralized agents can keep their survival while intervening rarely, locally, and mainly when the grid state makes an intervention necessary.

The mechanisms are therefore evaluated on two axes at once, survival and how often the agents leave the do-nothing action. A mechanism that improves sparsity by collapsing survival has not solved the problem, and neither has one that keeps survival by never becoming sparse.

Question. Can the agents keep high survival while behaving more like operators: acting rarely, acting locally, acting mainly when the grid state makes an intervention useful, and keeping the final policy decentralized?

Baseline and change. This block starts from the flat decentralized MAPPO topology-control actor. For agent i, the baseline policy is one categorical distribution over the full local action space:

$$\pi_i(a_i \mid o_i), \qquad a_i \in \{0, 1, \ldots, |\mathcal{A}_i| - 1\}.$$

Action 0 is the no-op action. During training, actions are sampled from the policy, while deterministic evaluation uses the greedy action by default:

$$a_{i,t}^{eval} = \arg\max_a \pi_i(a \mid o_{i,t}).$$

The sparse-control roadmap tests four ways to reduce unnecessary topology changes: evaluation-time rho overrides, an intervention-gated actor, fixed non-idle action penalties, and adaptive Lagrangian intervention budgets.

Motivation. The intervention gate is motivated by hierarchical topology-control work, where the action can be decomposed into "do nothing versus intervene", then where to act, then which local topology configuration to apply [4], [5]. In this thesis only the first level is adapted: each regional agent independently decides whether to do nothing or to execute one of its existing local topology actions. The fixed and adaptive sparse-control objectives are closer to constrained and budgeted RL: non-idle topology interventions are treated as a limited resource, following the general Lagrangian view of constrained policy optimization and budgeted control [6], [7], [8]. This is also operationally meaningful because switching frequency and topology deviation are real power-grid objectives, not just cosmetic metrics [9].

5.1 Evaluation-Time Rho Heuristics

Implementation detail. The rho heuristic is an evaluation-only diagnostic. It does not change the training objective and it does not prove that the policy learned sparse behavior. The policy first proposes the deterministic action $\bar{a}_{i,t}$, then the evaluator may replace it with action 0.

The global version uses the pre-action maximum line loading

$$\rho_t^{global} = \max_{\ell \in \mathcal{L}} \rho_{\ell,t},$$

and executes

$$a_{i,t}^{exec} = \begin{cases} 0, & \rho_t^{global} < \rho_{safe}, \\ \bar{a}_{i,t}, & \text{otherwise}, \end{cases} \qquad \rho_{safe} = 0.90.$$

The local version is decentralized. Agent i computes the maximum rho on the lines touching its regional substations,

$$\rho_{i,t}^{local} = \max_{\ell \in \mathcal{L}_i} \rho_{\ell,t},$$

and only that agent is forced to no-op when $\rho_{i,t}^{local} < \rho_{safe}$. A boundary line can appear in the local neighborhood of both adjacent agents, but no communication is required: both agents can observe the line through their own physical neighborhood.

Interpretation. The heuristic is useful to ask what happens if safe-state interventions are blocked, but it is not a learned method. Its action-0 rates must therefore be reported as final executed evaluation actions, not as evidence that the policy itself prefers action 0.

5.2 Intervention-Gated Actor

Implementation detail. The gated actor changes the actor factorization. Instead of one categorical head over all actions, each agent has a gate head and a non-idle action head:

$$\pi_i^{gate}(g_i \mid o_i), \qquad g_i \in \{0, 1\},$$

where 0 means do nothing and 1 means intervene, and

$$\pi_i^{nonidle}(b_i \mid o_i), \qquad b_i \in \{1, \ldots, |\mathcal{A}_i| - 1\}.$$

During training, the gate is sampled first. If $g_{i,t} = 0$, then $a_{i,t} = 0$. If $g_{i,t} = 1$, the final action is sampled from the non-idle head. The PPO log-probability of the executed action is therefore

$$\log P(a_i = 0) = \log P(g_i = 0),$$

and, for $a_i > 0$,

$$\log P(a_i) = \log P(g_i = 1) + \log P(b_i = a_i \mid g_i = 1).$$

Two deterministic decoders are used. The `final_action_map` decoder selects the most likely final executed action:

$$P(a_i = 0) = P(g_i = 0), \qquad P(a_i = a > 0) = P(g_i = 1)P(b_i = a \mid g_i = 1).$$

The `hierarchical_greedy` decoder first chooses the most likely gate decision, then chooses the most likely non-idle action only if the gate selects intervention.

The original coupled entropy term is

$$H_i = H(g_i) + P(g_i = 1)H(b_i).$$

This can accidentally reward intervention because larger $P(g_i = 1)$ unlocks more non-idle entropy. The separated entropy variant is

$$H_i = \alpha_{gate} H(g_i) + \alpha_{nonidle} H(b_i),$$

and is used in the gate-plus-budget diagnostic runs.

Interpretation. The gate is learned and decentralized, but it can restrict exploration more strongly than a reward penalty. If it collapses early to no-op, the non-idle head is rarely executed and receives little learning signal. For this reason, the gate is treated as an important diagnostic rather than the main sparse control contribution.

5.3 Fixed Sparse-Action Penalty

Implementation detail. The Sparse16 sweep adds a fixed reward penalty to the final executed action of each agent:

$$r'_{i,t} = r_{i,t} - \lambda_{fixed}\mathbf{1}[a_{i,t} \neq 0],$$

with

$$\lambda_{fixed} \in \{0.0, 0.001, 0.003, 0.01\}.$$

The grid is

$$\text{actor} \in \{\text{flat}, \text{gated}\}, \qquad \lambda_{fixed} \in \{0.0, 0.001, 0.003, 0.01\}, \qquad \text{seed} \in \{0, 1, 2\},$$

which gives $2 \times 4 \times 3 = 24$ runs. All Sparse16 runs set `safe_intervention_penalty = 0.0`, so the penalty is not rho-weighted.

Interpretation. Sparse16 changes only the training reward. It does not override actions during evaluation and it does not hard-mask exploration during training. The policy can still sample non-idle actions, but each such action must be useful enough to pay its fixed cost. The key comparison is `sparse16_flat_p010` versus `sparse16_gated_p010`, which asks whether the gate adds value once a simple sparse objective already exists.

5.4 Adaptive Intervention Budget

Implementation detail. The adaptive intervention budget turns sparse topology control into a local constraint. For each agent,

$$c_i(s_t, a_{i,t}) = \mathbf{1}[a_{i,t} \neq 0] w_i(s_t),$$

and the desired constraint is

$$\mathrm{E}_{\pi_i}\left[c_i(s_t, a_{i,t})\right] \leq d.$$

The implemented rollout reward removes the constant term that does not affect the PPO action gradient:

$$r'_{i,t} = r_{i,t} - \lambda_i c_i(s_t, a_{i,t}).$$

After each rollout, the local multiplier is updated by

$$\lambda_i \leftarrow \text{clip}\left(\lambda_i + \eta(\hat{C}_i - d), 0, \lambda_{\max}\right), \qquad \hat{C}_i = \frac{1}{T}\sum_{t=1}^{T} c_i(s_t, a_{i,t}).$$

If the observed cost $\hat{C}_i$ is above the budget d, future interventions become more expensive; if it is below the budget, the penalty relaxes. The default settings are `intervention_budget_lr = 0.02`, `intervention_budget_init_lambda = 0.0`, and `intervention_budget_max_lambda = 10.0`.

Three cost modes are used:

- **Non-idle cost:** $w_i(s_t) = 1$, so $c_i(s_t, a_{i,t}) = \mathbf{1}[a_{i,t} \neq 0]$. This is an adaptive version of a plain sparse-action penalty.
- **Global-safe cost:** $w^{global}(s_t) = \sigma(k(\rho_{safe} - \rho_t^{global}))$. This penalizes non-idle actions mostly when the whole grid is safe.
- **Local-safe cost:** $w_i^{local}(s_t) = \sigma(k(\rho_{safe} - \rho_{i,t}^{local}))$. This is the decentralized version: safe local states make interventions expensive, while stressed local states make them cheap.

In the AIB setting, $\rho_{safe} = 0.90$ is not a hard action override. It is the center of a smooth sigmoid safety weight.

AIB experiment grid. The first AIB folder, `configs/adaptive_intervention_budget_7`, contains the main local-budget ablations:

- `aib_00_flat_local_t020`: flat actor, local-safe cost, $d = 0.20$;
- `aib_01_flat_local_t010`: stricter local-safe budget, $d = 0.10$;
- `aib_02_flat_local_t035`: looser local-safe budget, $d = 0.35$;
- `aib_03_gate_hgreedy_sep_local_t020`: gated actor, hierarchical-greedy evaluation, separated entropy, local-safe cost, $d = 0.20$;
- `aib_04_flat_nonidle_t020`: flat actor with adaptive plain non-idle cost, $d = 0.20$.

The comparison `aib_00_flat_local_t020` versus `aib_04_flat_nonidle_t020` isolates the value of local rho weighting.

The mechanism-ablation folder `configs/adaptive_intervention_budget_mechanism_15` contains 15 additional flat-actor runs. It separates fixed versus adaptive coefficients, plain non-idle versus state-dependent costs, global versus local rho weighting, and rho-threshold sensitivity.

Table 5.1: Sparse-control mechanisms and where they affect the policy.

Method	Training effect	Evaluation effect	Main risk
Baseline	Samples from the flat policy	Greedy argmax by default	Standard entropy-controlled exploration
Rho heuristic	No training change	Hard no-op override in safe states	Evaluation is manually changed
Intervention gate	Changes action factorization and sampling	Final-action MAP or hierarchical greedy	Gate collapse can starve non-idle exploration
Sparse16	Fixed reward penalty on non-idle actions	No action override	Penalty coefficient must be tuned
AIB non-idle	Adaptive reward penalty on non-idle actions	No action override	Multiplier can grow if target is too strict
AIB local-safe	Adaptive state-dependent reward penalty	No action override	Depends on the local rho safety-weight design

Table 5.2: Main sparse-control comparisons to report.

Question	Comparison
Does a fixed sparse penalty already work?	Baseline versus `sparse16_flat_p010`
Does the gate help beyond a sparse objective?	`sparse16_flat_p010` versus `sparse16_gated_p010`
Fixed versus adaptive coefficient?	`sparse16_flat_p010` or `aibm_00_fixed_nonidle_p006` versus `aib_04_flat_nonidle_t020`
Does local-safe weighting matter?	`aib_00_flat_local_t020` versus `aib_04_flat_nonidle_t020`
Does global-safe weighting matter?	`aibm_02_adaptive_global_t020` versus `aib_00_flat_local_t020`
Is $\rho_{safe} = 0.90$ special?	`aibm_03` with 0.85, `aib_00` with 0.90, and `aibm_04` with 0.95
Is the heuristic only an evaluation trick?	Rho-heuristic runs versus learned sparse/AIB runs

Full-test checkpoint summaries. The following tables report the average full-test episodic survival over the three evaluated seeds. The survival values are computed directly from the JSON files in `Topology_Task/outputs/full_test_eval`. These JSON files also record where the compact action-distribution artifacts were written. The action-0 fractions are computed from the corresponding local `Topology_Task/outputs/full_test_eval_actions/*/action_summary.json` files and averaged over

seeds. The all-agent action-0 column is the arithmetic mean of the three per-agent action-0 fractions. The do-nothing row is a reference policy that always selects action 0; its survival is computed from `Topology_Task/outputs/do_nothing_eval/bus14_test_20`. Remaining "–" entries indicate that the compact action summaries for those full-test jobs are not available locally.

Table 5.3: Full-test summary for evaluation-time rho heuristics.

Run type	Avg. episodic survival (%)	Avg. action-0 all agents	Avg. action-0 agent 0	Avg. action-0 agent 1	Avg. action-0 agent 2
Do-nothing policy	20.25	1.000	1.000	1.000	1.000
Baseline flat policy	93.67	0.434	0.554	0.385	0.363
Global rho heuristic, $\rho_{safe} = 0.90$	98.36	0.936	0.933	0.936	0.940
Local rho heuristic, $\rho_{safe} = 0.90$	97.07	0.968	0.992	0.985	0.928

Table 5.4: Full-test summary for intervention-gated actors.

Run type	Avg. episodic survival (%)	Avg. action-0 all agents	Avg. action-0 agent 0	Avg. action-0 agent 1	Avg. action-0 agent 2
Do-nothing policy	20.25	1.000	1.000	1.000	1.000
Baseline flat policy	93.67	0.434	0.554	0.385	0.363
Intervention gate, final-action MAP	89.82	0.802	0.826	0.889	0.690
Intervention gate, hierarchical greedy	93.66	0.395	0.066	0.844	0.276

Table 5.5: Full-test summary for Sparse16 flat-policy runs.

Run type	Avg. episodic survival (%)	Avg. action-0 all agents	Avg. action-0 agent 0	Avg. action-0 agent 1	Avg. action-0 agent 2
Do-nothing policy	20.25	1.000	1.000	1.000	1.000
Baseline flat policy	93.67	0.434	0.554	0.385	0.363
Flat Sparse16, $\lambda_{fixed} = 0.000$	98.23	0.494	0.524	0.463	0.496
Flat Sparse16, $\lambda_{fixed} = 0.001$	96.75	0.486	0.642	0.401	0.416
Flat Sparse16, $\lambda_{fixed} = 0.003$	93.97	0.575	0.691	0.346	0.688
Flat Sparse16, $\lambda_{fixed} = 0.010$	97.44	0.905	0.907	0.954	0.853

Metrics to report. The main performance metric remains episodic survival. It should be paired with sparsity and physical-diagnostic metrics: per-agent action-0 fractions, per-agent non-idle rates, the distribution of how many agents act simultaneously, intervention-budget multipliers, budget costs and violations, the rate of interventions in costly safe states, pre- and post-action max rho, and topology-distance changes.

Interpretation. The current roadmap interpretation is that the gate is not the strongest contribution by itself. Sparse reward shaping already improves action-0 usage, the adaptive budget makes the sparse penalty self-tuning, and the local-safe cost makes the penalty more physically meaningful. In particular, `sparse16_flat_p010` is a strong fixed-penalty baseline, while the most principled learned sparse-control candidate is `aib_00_flat_local_t020`: it keeps the original decentralized actor, does not rely on an evaluation-time override, learns the intervention penalty adaptively, and uses a local state-dependent safety weight.

Table 5.6: Full-test summary for Sparse16 gated-policy runs.

Run type	Avg. episodic survival (%)	Avg. action-0 all agents	Avg. action-0 agent 0	Avg. action-0 agent 1	Avg. action-0 agent 2
Do-nothing policy	20.25	1.000	1.000	1.000	1.000
Baseline flat policy	93.67	0.434	0.554	0.385	0.363
Gated Sparse16, $\lambda_{fixed} = 0.000$	78.51	0.787	0.747	0.949	0.665
Gated Sparse16, $\lambda_{fixed} = 0.001$	73.82	0.862	0.956	0.832	0.799
Gated Sparse16, $\lambda_{fixed} = 0.003$	94.59	0.887	0.934	0.929	0.799
Gated Sparse16, $\lambda_{fixed} = 0.010$	85.69	0.974	0.990	0.984	0.947

Table 5.7: Full-test summary for adaptive intervention budget runs.

Run type	Avg. episodic survival (%)	Avg. action-0 all agents	Avg. action-0 agent 0	Avg. action-0 agent 1	Avg. action-0 agent 2
Do-nothing policy	20.25	1.000	1.000	1.000	1.000
Baseline flat policy	93.67	0.434	0.554	0.385	0.363
Flat local-safe AIB, $d = 0.20$	98.31	0.978	0.980	0.989	0.966
Flat local-safe AIB, $d = 0.10$	98.39	0.979	0.992	0.995	0.949
Flat local-safe AIB, $d = 0.35$	96.61	0.934	0.959	0.947	0.896
Gated local-safe AIB, $d = 0.20$, hierarchical greedy	33.87	0.999	1.000	1.000	0.998
Flat non-idle AIB, $d = 0.20$	95.08	0.986	0.997	0.992	0.970

Table 5.8: Full-test summary for the teacher-student distillation runs.

Run type	Avg. episodic survival (%)	Avg. action-0 all agents	Avg. action-0 agent 0	Avg. action-0 agent 1	Avg. action-0 agent 2
Do-nothing policy	20.25	1.000	1.000	1.000	1.000
Teacher: MAPPO with local rho heuristic	97.07	0.968	0.992	0.985	0.928
Student BC, balanced non-idle 0.50, weight 5, aux 0.50	92.14	0.939	0.974	0.925	0.919
Student BC, balanced non-idle 0.20, weight 3, aux 0.25	90.76	0.957	0.982	0.948	0.940

Chapter 6

Transfer to the 36-Substation WCCI Grid

Every result so far was obtained on `bus14`. That grid is small enough to iterate on but too small to claim anything about scale: fourteen substations, twenty lines, three agents. This chapter moves to the L2RPN WCCI 2020 grid, more than twice as many substations and three times as many lines, and asks the question that motivates a graph encoder in the first place. If the actor reads the grid as a graph rather than as a flat vector, its weights are independent of how many substations exist, and a representation learned on `bus14` should carry over.

Before running that transfer, the two environments have to be made comparable and the encoder's inputs have to be checked. This chapter does both. The comparison turns out to matter more than expected: the graph features the encoder consumes are raw physical quantities, and they do not follow the same distribution on the two grids.

6.1 The target environment

Table 6.1 contrasts the source and target grids. The agent partitions follow the same principle in both cases, contiguous electrical areas, but WCCI is split into four regions rather than three, and the regions are markedly unequal: one agent controls seventeen substations while another controls five.

Table 6.1: Source and target environments. Both use difficulty 0, topology actions, decentralized agents and an 80/20 train/test chronic split with split seed 0.

	`bus14`	`bus36_wcci_nomaint`
Grid2Op environment	`l2rpn_case14_sandbox`	`l2rpn_wcci_2020`
Substations	14	36
Lines	20	59
Agents	3	4
Chronics	1,004	2,880
Episode length	up to 8,064 steps	up to 8,062 steps
Maintenance	none	disabled (Section 6.1.1)

6.1.1 Removing maintenance

Of the environments Grid2Op distributes, `l2rpn_case14_sandbox` is the only one above the toy scale that ships without scheduled maintenance. Every larger environment, WCCI 2020 included, injects planned line outages. That is a problem for a transfer study: a policy trained without maintenance and evaluated with it faces a hazard it has never seen, and the two observation spaces differ as well, since the maintenance countdown features are only present when maintenance exists.

The two settings are therefore aligned by removing maintenance from WCCI rather than inventing a schedule for `bus14`, for which Grid2Op provides no specification. WCCI generates maintenance at every reset from a `maintenance_meta.json` per chronic, through the chronics class `GridStateFromFileWithForecastsWithMaintenance`. Overriding that class with `GridStateFromFileWithForecasts`, the class `l2rpn_case14_sandbox` already uses, removes the outages at the source while leaving load and generation series untouched. The resulting scenario is registered as `bus36_wcci_nomaint`; it differs from `bus36_wcci` in exactly this one respect.

The effect is large. On a fixed sample of fifty held-out chronics, evaluated by name so that both settings see the same episodes, a do-nothing policy survives 21.7% of the horizon with maintenance and 56.0% without, and completes 6% of episodes with maintenance against 52% without. Maintenance was responsible for most of the failures on this grid.

6.1.2 Action space

The unreduced WCCI action space is not trainable. Enumerating `change_bus` and `change_line_status` over each agent's region gives the sizes in the first column of Table 6.2: nearly 67,000 actions in total, of which agent 1 alone accounts for 65,642. All experiments therefore use the reduced action space obtained by the brute-force outcome procedure, at $k = 256$ per agent.

Table 6.2: Per-agent action-space sizes on `bus36_wcci_nomaint`. Agents 0 and 2 are already exhaustive at $k = 256$, so the reduction only binds on agents 1 and 3.

Agent	Substations	Full	Reduced ($k = 256$)
`agent_0`	5	77	77
`agent_1`	5	65,642	256
`agent_2`	9	127	127
`agent_3`	17	1,119	256
Total	36	66,965	716

The choice of k was made by evaluating a one-step simulator-greedy policy over each candidate reduction on the same fifty chronics (Table 6.3). Performance saturates at $k = 256$: the difference between $k = 256$ and $k = 1024$ is four chronics won against three lost, which is noise, while both clearly beat $k \leq 128$. Since $k = 256$ is the smallest reduction in the saturated group, it gives the smallest action heads for no measurable loss in what the action set can achieve.

Table 6.3: One-step simulator-greedy policy over each reduced action space, on the same fifty held-out chronics of `bus36_wcci_nomaint`. The do-nothing row is the common reference; no reduction was evaluated against a different chronic set.

Policy	Mean survival	Full-episode survival	Episodes worse than do-nothing
Do nothing	0.560	52%	—
Greedy, $k = 64$	0.819	64%	3
Greedy, $k = 128$	0.847	76%	3
Greedy, $k = 256$	**0.976**	**92%**	0
Greedy, $k = 512$	0.883	82%	2
Greedy, $k = 1024$	0.953	90%	0

These two rows bracket the transfer experiments. A learned policy that lands near 0.56 mean survival has learned nothing beyond doing nothing; one approaching 0.9 is doing real work. The greedy row is not a strict upper bound, since it queries the simulator at every decision step and is myopic, but it is a strong reference.

6.2 Why the input distribution matters

Only the graph encoder can cross grids. The per-agent action heads cannot: the partitions differ in size and count, so no weight in a head has a counterpart on the other grid. The centralized critic reads a flat, grid-sized observation and is likewise grid-specific. What transfers is the encoder, and what the encoder reads is the node and edge feature vectors.

Those vectors are unnormalized. The configuration flag `norm_obs` is enabled, but it standardizes the *flat* observation, on a code path that runs before and separately from graph construction, and the flat observation feeds only the critic, since `gnn_concat_flat` is disabled. The graph tensors pass through a feature processor whose two mechanisms, deterministic physical scaling and running standardization, are both switched off throughout the screening protocol; with both disabled the processor returns its input unchanged.

A frozen `bus14` encoder therefore receives WCCI's raw physical quantities. Whether that is a problem is an empirical question about how similar the two distributions are.

6.3 Measurement protocol

Both environments were rolled out under a do-nothing policy for 6,000 environment steps, with a cap of 300 steps per chronic so the sample spreads over roughly twenty scenarios per grid rather than concentrating in one. At each step the per-agent local graphs were rebuilt and every node and active physical edge recorded, pooling agents together because a single encoder is shared across them. This yields 276,000 node observations for `bus14` and a capped 400,000 for WCCI, and 312,000 and 400,000 edge observations respectively.

Four statistics summarize each feature. Writing F_A and F_B for the two empirical distributions with means μ_A, μ_B and standard deviations σ_A, σ_B, and $\sigma_p = \sqrt{(\sigma_A^2 + \sigma_B^2)/2}$ for their pooled spread:

$$\text{offset} = \frac{\mu_B - \mu_A}{\sigma_p}, \qquad \text{scale ratio} = \frac{\sigma_B}{\sigma_A}, \tag{6.1}$$

$$\text{KS} = \sup_x |F_A(x) - F_B(x)|, \qquad W_1/\sigma_p = \frac{1}{\sigma_p} \int_0^1 |F_A^{-1}(q) - F_B^{-1}(q)| \, \mathrm{d}q. \tag{6.2}$$

The pairing is deliberate. The offset is blind to a change in spread and the scale ratio is blind to a displacement, so a feature is only unproblematic when both are near their neutral values. KS measures disagreement in the bulk of the distribution and is comparatively insensitive to tails, which W_1 does weigh.

6.4 What the encoder reads

The `bus` graph represents each busbar as a node and each transmission line as an edge. Table 6.4 lists the ten channels.

Table 6.4: Node and edge features of the `bus` graph, with the fraction of sampled entries equal to zero on each grid. A busbar only carries generation or load when an asset is attached to it, which makes four of the six node channels structurally sparse.

Feature	Meaning	Unit	Zeros (%)	
			`bus14`	WCCI
Node features				
`gen_p`	Active power injected by attached generators	MW	84.0	88.0
`gen_theta`	Voltage angle at the generator connection	deg	84.8	80.3
`load_p`	Active power drawn by attached loads	MW	56.5	56.4
`load_theta`	Voltage angle at the load connection	deg	56.5	58.0
`time_before_cooldown_sub`	Steps until this substation may be reconfigured	steps	100	100
`domain_mask`	1 for the agent's own substations, 0 for borrowed neighbors	—	39.1	35.7
Edge features				
`line_status`	1 connected, 0 disconnected	—	0	0
`rho`	Current flow divided by thermal limit	p.u.	0	0
`timestep_overflow`	Consecutive steps above the thermal limit	steps	100	100
`time_before_cooldown_line`	Steps until this line may be switched	steps	100	100

One feature is dimensionless by construction. `rho` divides each line's current flow by that line's own thermal limit, so it is expressed in units the grid itself defines. Every other physical channel carries the units of the grid it came from.

Three channels are constant across the whole sample and carry no information here. Under a do-nothing policy no cooldown is ever triggered, and only active physical edges are recorded, which makes `line_status` identically one. They are reported for completeness and excluded from the discussion.

6.5 Results

6.5.1 Aggregate statistics

Table 6.5 reports the four statistics over all sampled entries.

Table 6.5: Distribution statistics over all sampled nodes and edges, ordered by KS. Offset and W_1 are in pooled standard deviations; the scale ratio is WCCI over `bus14`. Constant channels give undefined ratios.

Feature	μ_{bus14}	μ_{WCCI}	Offset	Scale ratio	KS	W_1/σ_p
`load_theta`	−2.402	0.614	+1.024	0.850	0.331	1.024
`rho`	0.364	0.319	−0.260	1.048	0.141	0.261
`gen_theta`	−0.633	0.215	+0.503	0.919	0.115	0.503
`gen_p`	8.645	9.451	+0.020	2.435	0.089	0.266
`load_p`	9.723	7.579	−0.063	2.363	0.040	0.238
`domain_mask`	0.609	0.643	+0.071	0.982	0.034	0.070
`timestep_overflow`	5.1e-5	7.5e-6	−0.007	0.403	0.000	0.000
`line_status`	1	1	—	—	0.000	—
`time_before_cooldown_line`	0	0	—	—	0.000	—
`time_before_cooldown_sub`	0	0	—	—	0.000	—

Read carelessly this table looks mild. Only `load_theta` exceeds KS = 0.2, and the power channels sit near KS = 0.05. But their scale ratios are 2.4, and that is exactly the pattern the offset–scale pairing was introduced to catch: the two distributions share a centre while differing in width, so KS stays small because the bulk still overlaps.

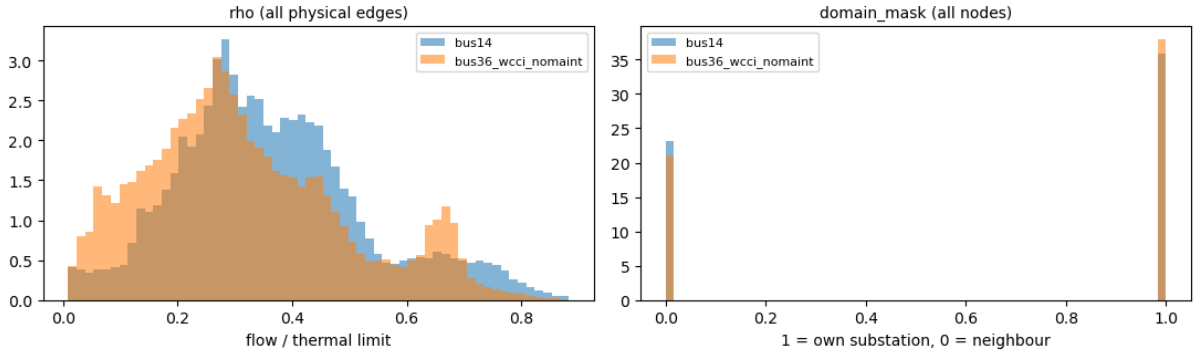


Figure 6.1: The two dense channels. `rho` is nearly identical on the two grids, as expected from a quantity already normalized by each grid's own thermal limits. `domain_mask` is structural, and its small difference only reflects how many one-hop neighbor nodes each agent partition pulls in.

6.5.2 Conditioning on the nodes that carry signal

The aggregate table understates the problem, because most of its rows are structural zeros. A busbar with no generator contributes a zero to `gen_p` on both grids alike, and roughly 85% of rows are of that kind. Statistics computed over such a column largely describe how many zeros it contains.

Restricting each sparse channel to the nodes where the corresponding asset is actually attached gives Table 6.6. Every statistic roughly doubles.

Table 6.6: The four sparse node channels, restricted to nodes that carry the corresponding asset. Means and standard deviations are in MW for the power channels and degrees for the angles.

	bus14		WCCI					
Feature	μ	σ	μ	σ	Offset	Scale ratio	KS	W_1/σ_p
`load_theta`	−5.53	2.43	1.46	4.01	+2.106	1.648	**0.758**	2.106
`gen_theta`	−4.16	2.36	1.10	3.50	+1.757	1.482	**0.724**	1.757
`gen_p`	53.96	23.12	78.75	134.67	+0.257	**5.824**	0.496	0.744
`load_p`	22.36	22.87	17.37	65.69	−0.101	2.873	0.093	0.378

A KS of 0.758 means the two empirical CDFs differ by 76 percentage points at their widest separation. These are not the same distribution in any useful sense. The angle means state it plainly: `bus14` sits near −5.5° and −4.2° while WCCI sits near +1.5° and +1.1°, a systematic displacement of five to seven degrees. Generation conditioned on actual generators averages 54 ± 23 MW on `bus14` against 79 ± 135 MW on WCCI.

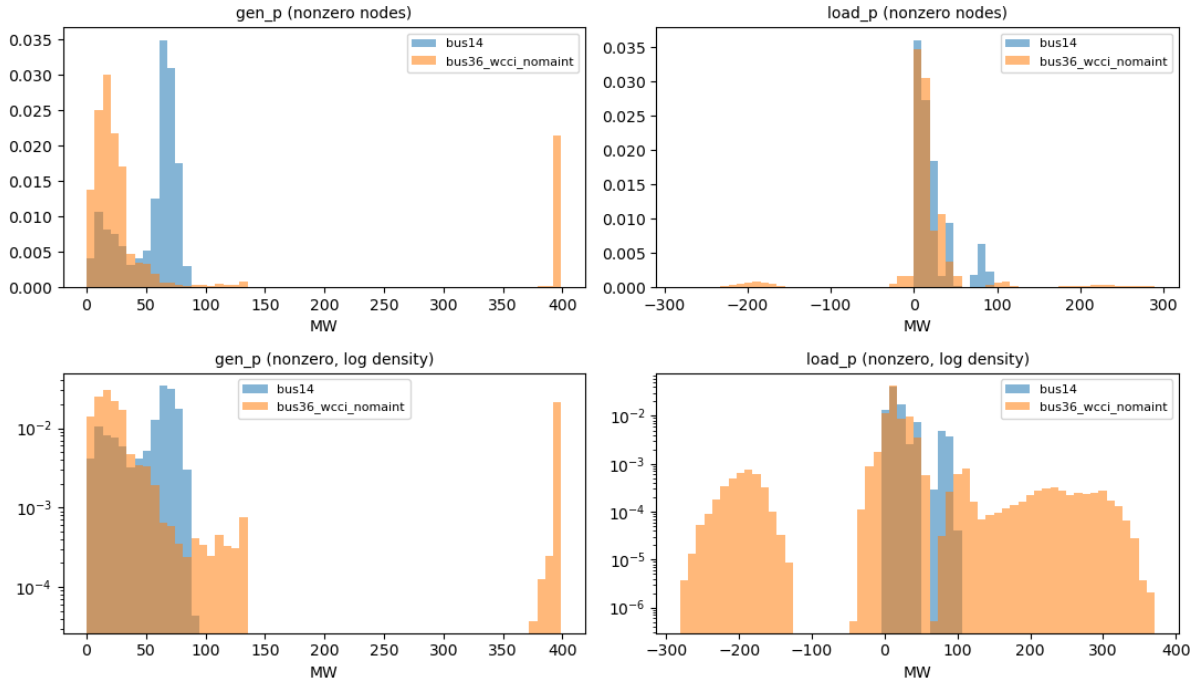


Figure 6.2: Power channels at nodes carrying the corresponding asset, on a linear density scale (top) and a logarithmic one (bottom). The logarithmic row is the informative one: WCCI carries a long right tail that `bus14` does not have, reaching 399 MW of generation against a 92 MW ceiling, and `load_p` extends below zero on 3.5% of WCCI load nodes while `bus14` never produces a negative value.

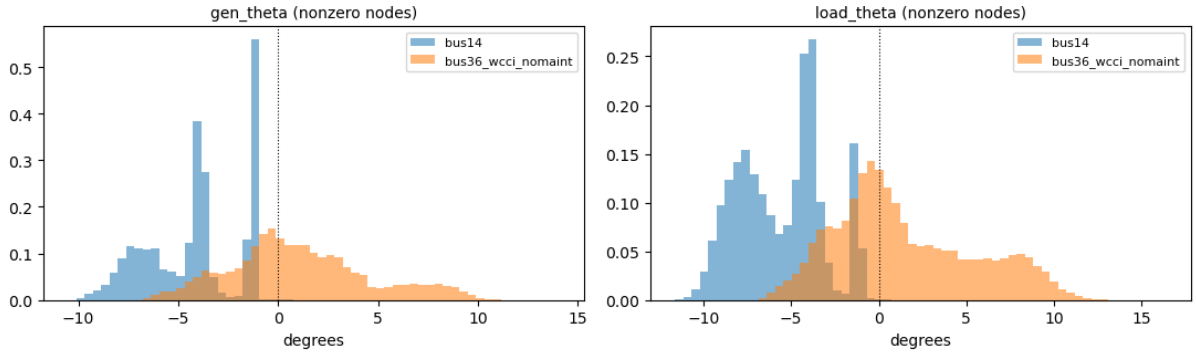


Figure 6.3: Voltage angles at nodes carrying the corresponding asset. The `bus14` distributions lie almost entirely on the negative side, range $[-11.6°, +1.5°]$, while WCCI spreads across both signs out to $+16.4°$. The distributions are displaced rather than merely rescaled.

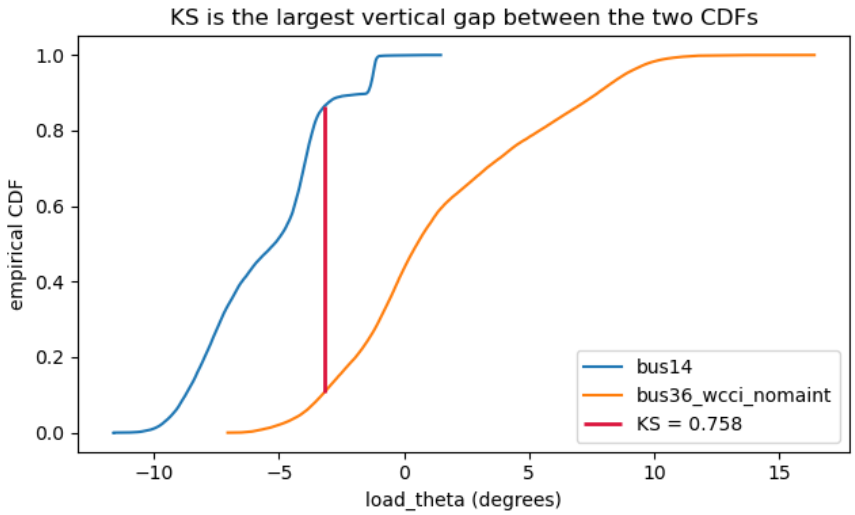


Figure 6.4: The empirical CDFs of `load_theta` at load-carrying nodes. The KS statistic is the largest vertical distance between the two curves, marked in red.

6.6 Two failure modes

Figure 6.5 separates the channels along the two axes. The distinction is not cosmetic, because the architecture responds to them differently.

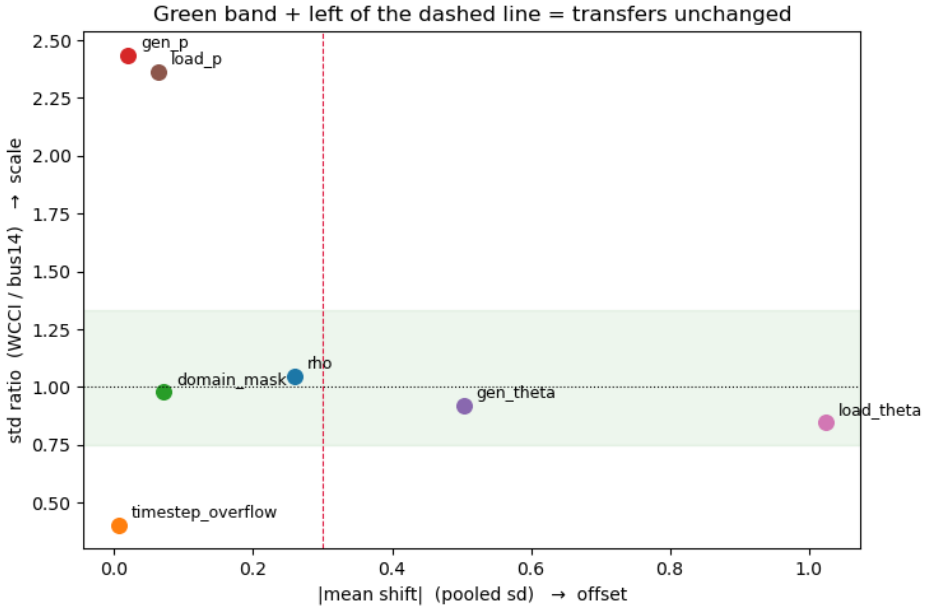


Figure 6.5: Absolute offset against scale ratio, over all sampled entries. The shaded band marks a scale ratio within a third of unity and the dashed line an offset of 0.3 pooled standard deviations. Channels outside those bounds fail in one of two distinct ways.

The actor's node pre-encoder is a linear map followed by ReLU and LayerNorm. LayerNorm absorbs much of a uniform change of *scale*: multiplying every input channel by a constant leaves the normalized activations largely unchanged. It does not absorb an *offset*. Displacing a channel by a constant moves the pre-activations to a different region of the ReLU, changing which units fire at all, and normalization applied afterwards cannot undo that.

This makes the angles, not the powers, the binding problem. The power channels differ in width by

factors of 2.9 to 5.8, which the architecture partially self-corrects, though the negative `load_p` region on WCCI reaching −281 MW is genuinely unseen territory rather than a wider version of something familiar. The angles differ by roughly two pooled standard deviations of displacement, which it does not correct at all.

`rho`, meanwhile, needs no correction: a scale ratio of 1.048 and an offset of −0.26. The single most decision-relevant channel, the one that says how close a line is to its limit, is already comparable across grids because it was defined in per-unit terms. That is an argument for expressing features in grid-relative units wherever the physics permits it.

6.7 Implication for the transfer experiments

A frozen `bus14` encoder evaluated on WCCI is being asked to interpret inputs displaced by about two standard deviations on two of its six node channels, and widened several-fold on two more. Any deficit such a policy shows relative to one trained from scratch is therefore confounded: it may reflect a representation that does not generalize, or merely a representation reading miscalibrated inputs.

Of the two available preprocessing mechanisms, only one addresses the diagnosis. Deterministic physical scaling divides angle features by a fixed constant of 180, identical on both grids, so it rescales both sides equally and leaves the displacement exactly where it was; it would fix the power widths, since that divisor is the per-environment maximum generator capacity. Running standardization maintains per-environment, per-node-type statistics over a feature set that includes both power channels and both angle channels while deliberately excluding `rho`, which the measurements above confirm needs no help.

Running standardization is therefore the mechanism that matches the measured problem, and on these numbers it reads as a prerequisite for a clean transfer result rather than an optional refinement. The cost is that the source checkpoints must be retrained under the same setting: a frozen encoder's weights are bound to the input scale they were trained on, so a source trained on raw features cannot be dropped into a normalized target.

Appendix A

GINE and GAT Graph-Actor Architectures

This appendix specifies the complete edge-aware actor used in the graph experiments. GINE and GAT receive the same graph inputs and use the same pooling and action head. Their only architectural difference is the message-passing operator: GINE inserts an edge embedding into the message content, whereas GAT uses an edge embedding to decide how strongly the message is weighted.

A.1 Shared input and encoder pipeline

For agent a, the encoder receives the local graph $\mathcal{G}_t^{(a)}$. Candidate edges with `edge_mask`$= 0$ are removed before message passing, so the layer sees only the active directed set $\mathcal{E}_t^{(a)}$. The source of $u \to v$ sends a message and v aggregates it. Node and edge masks do not become learned features.

Each raw node vector $\mathbf{x}_v$ first passes through

$$\mathbf{h}_v^{(0)} = \mathrm{LayerNorm}(\mathrm{ReLU}(\mathbf{W}_x \mathbf{x}_v + \mathbf{b}_x)), \tag{A.1}$$

which produces a 128-dimensional node state. Learned global node-identifier embeddings and the optional external edge pre-encoder are disabled. Every GINE or GAT layer nevertheless contains its own trainable projection of the edge vector to the dimension required by that layer.

Two message-passing layers are used. After each layer, the implementation applies ReLU followed by LayerNorm:

$$\mathbf{h}_v^{(k+1)} = \mathrm{LayerNorm}\Big(\mathrm{ReLU}(\widetilde{\mathbf{h}}_v^{(k+1)})\Big). \tag{A.2}$$

The same encoder parameters are shared by all agents. Each agent nevertheless passes its own local graph, with its own node set, active edges, and one-hop context, through that shared encoder.

A.2 GINE message passing

For an active edge $u \to v$, layer k projects the complete edge feature vector and adds it to the sender state:

$$\widetilde{\mathbf{e}}_{uv}^{(k)} = \mathbf{W}_e^{(k)} \mathbf{e}_{uv} + \mathbf{b}_e^{(k)}, \tag{A.3}$$

$$\mathbf{m}_{u\to v}^{(k)} = \mathrm{ReLU}\Big(\mathbf{h}_u^{(k)} + \widetilde{\mathbf{e}}_{uv}^{(k)}\Big), \tag{A.4}$$

$$\widetilde{\mathbf{h}}_v^{(k+1)} = \mathrm{MLP}^{(k)}\left((1+\epsilon)\mathbf{h}_v^{(k)} + \sum_{u:(u,v)\in\mathcal{E}_t} \mathbf{m}_{u\to v}^{(k)}\right). \tag{A.5}$$

The implementation uses the PyTorch Geometric default $\epsilon = 0$ without a trainable epsilon parameter. Each internal GINE MLP is Linear–ReLU–Linear and outputs 128 features. Equation A.5 shows why

edge attributes are part of the message content. Two identical node states can send different messages when one edge is an overloaded physical line and the other is an asset-attachment relation. Likewise, reversing a typed heterogeneous edge activates a different relation-one-hot column and can produce a different message.

GINE uses an unweighted sum over incoming active edges. It does not learn a softmax competition between neighbors. The edge embedding changes what is sent; the number of incoming messages and their vector sum determine the aggregate.

A.3 GAT message passing

GAT first computes a score for every active incoming edge and attention head. For head r, the implementation has the form

$$s_{uv}^{(k,r)} = \mathrm{LeakyReLU}\Big(\mathbf{a}_{s,r}^{\top}\mathbf{W}_{s,r}\mathbf{h}_u^{(k)} + \mathbf{a}_{d,r}^{\top}\mathbf{W}_{d,r}\mathbf{h}_v^{(k)} + \mathbf{a}_{e,r}^{\top}\mathbf{W}_{e,r}\mathbf{e}_{uv}\Big), \tag{A.6}$$

$$\alpha_{uv}^{(k,r)} = \frac{\exp s_{uv}^{(k,r)}}{\sum_{w:(w,v)\in\mathcal{E}_t} \exp s_{wv}^{(k,r)}}, \tag{A.7}$$

$$\widetilde{\mathbf{h}}_v^{(k+1)} = \frac{1}{H}\sum_{r=1}^{H}\sum_{u:(u,v)\in\mathcal{E}_t} \alpha_{uv}^{(k,r)}\mathbf{W}_r\mathbf{h}_u^{(k)}. \tag{A.8}$$

GAT uses $H = 4$ heads and `concat=false`, so the four head outputs are averaged and the layer still returns 128 features. The edge embedding affects α_{uv}: it changes how much attention the receiver assigns to the sender. In the standard GAT operator used here, the edge vector is not added to the value vector $\mathbf{W}_r\mathbf{h}_u$. This is the central difference from GINE.

GAT internally adds a self-loop for every node. That computational self-loop is separate from the optional explicit `relation_self` edge in the graph schema.

Table A.1: Exact architectural difference between the implemented GINE and GAT actors.

Component	GINE	GAT
Edge use	Added to the sender state inside every message	Enters the attention score for every incoming edge
Neighbor aggregation	Unweighted sum	Softmax-weighted sum
Self information	Explicit $(1+\epsilon)\mathbf{h}_v$ term	Internal GAT self-loop
Multiple heads	Not used	Four heads, averaged rather than concatenated
Post-layer operation	ReLU, then LayerNorm	ReLU, then LayerNorm
Pooling and action head	Shared implementation	Shared implementation

A.4 Pooling and graph output

With mean pooling, the graph representation after the second message-passing layer is

$$\overline{\mathbf{h}}_G = \frac{\sum_v m_v\mathbf{h}_v^{(K)}}{\max(1, \sum_v m_v)}, \tag{A.9}$$

where m_v is the valid-node mask. This is one mean over all valid node types. It is not a separate mean per type. Therefore:

- the busbar graph pools all busbars;
- the direct heterogeneous graph pools busbars, generators, and loads;
- the explicit-line graph also pools transmission-line nodes.

For a local actor graph, one-hop contextual nodes are included in the mean. The ordinary GINE/GAT mean readout does not restrict pooling to controlled nodes. Sum and maximum pooling change only Equation A.9; they do not change message passing.

The pooled 128-dimensional vector then passes through the shared graph output projection

$$\mathbf{z}_a = \mathrm{ReLU}(\mathbf{W}_o \overline{\mathbf{h}}_G + \mathbf{b}_o), \qquad \mathbf{z}_a \in \mathbb{R}^{128}. \tag{A.10}$$

When virtual-node readout is selected, the final virtual-node state replaces $\overline{\mathbf{h}}_G$; the output projection and its dimension stay the same.

A.5 Agent-specific action head

The flat observation is not concatenated to $\mathbf{z}_a$. Each agent has its own two-layer MLP action head:

$$\mathbf{q}_a^{(1)} = \mathrm{ReLU}(\mathbf{W}_a^{(1)} \mathbf{z}_a + \mathbf{b}_a^{(1)}), \tag{A.11}$$

$$\mathbf{q}_a^{(2)} = \mathrm{ReLU}(\mathbf{W}_a^{(2)} \mathbf{q}_a^{(1)} + \mathbf{b}_a^{(2)}), \tag{A.12}$$

$$\boldsymbol{\ell}_a = \mathbf{W}_a^{(3)} \mathbf{q}_a^{(2)} + \mathbf{b}_a^{(3)}, \qquad \boldsymbol{\ell}_a \in \mathbb{R}^{|\mathcal{A}_a|}, \tag{A.13}$$

$$\pi_a(\cdot \mid \mathcal{G}_t^{(a)}) = \mathrm{Categorical}(\mathrm{logits} = \boldsymbol{\ell}_a). \tag{A.14}$$

Both hidden layers contain 128 units. During training, an action is sampled from this categorical distribution. Deterministic evaluation selects $\arg\max_j \ell_{a,j}$. Action zero is the local do-nothing action.

Only the GNN encoder and graph output projection are shared across agents. The action heads are agent-specific because agents can have different local action sets and therefore different output widths $|\mathcal{A}_a|$. The critic is a separate MLP over the centralized flat state and does not reuse the actor GINE/GAT embedding.

Equation A.14 is the default head. The optional candidate-action head of Section 3.5 keeps this pipeline up to $\mathbf{z}_a$ and additionally consumes the post-message-passing node states $\mathbf{h}_v^{(K)}$, which are the same states that Equation A.9 averages.

A.6 Exact node inputs by representation

All node types of one representation receive the same feature columns. Columns not listed for a node type are zero. Type identity reaches GINE and GAT through the `is_*` columns.

Table A.2: Node types and nonzero input features received by GINE and GAT.

Representation	Node type	Populated input features
`bus`	Busbar	`gen_p`, `gen_theta`, `load_p`, `load_theta`, `time_before_cooldown_sub`, `domain_mask`
`heterogeneous`	Busbar	`is_busbar`, `time_before_cooldown_sub`, `connected`, `domain_mask`
	Generator	`is_generator`, `p=gen_p`, `theta=gen_theta`, `connected`, `domain_mask`
	Load	`is_load`, `p=load_p`, `theta=load_theta`, `connected`, `domain_mask`
`heterogeneous_line`	Busbar	`is_busbar`, `time_before_cooldown_sub`, `connected`, `domain_mask`
	Generator	`is_generator`, `p=gen_p`, `theta=gen_theta`, `connected`, `domain_mask`
	Load	`is_load`, `p=load_p`, `theta=load_theta`, `connected`, `domain_mask`
	Transmission line	`is_transmission_line`, `line_status`, `rho`, `timestep_overflow`, `time_before_cooldown_line`, optional maintenance times, `connected`, `domain_mask`
Any augmented schema	Substation summary	Learned substation-indexed vector; no raw physical feature row
	Virtual node	One learned vector shared across graphs; no raw physical feature row

A.7 Exact edge inputs by representation

The edge vector contains all edge information consumed by GINE and GAT. Inactive candidates are filtered out before the vectors in Table A.3 reach a convolution.

Table A.3: Directed edge relations and edge-feature vectors received by GINE and GAT. "Relation" denotes one active relation-one-hot column.

Representation	Directed relation	Edge features
`bus`	Physical busbar ↔ busbar	`line_status`, `rho`, `timestep_overflow`, `time_before_cooldown_line`, optional maintenance times; no relation one-hot
	Optional self or same-substation	Same width; physical values zero and `rho`= 1; no relation one-hot
`heterogeneous`	Physical busbar ↔ busbar	Complete measured physical-line block plus `relation_physical_line`
	Generator ↔ busbar	Neutral physical block plus direction-specific generator relation
	Load ↔ busbar	Neutral physical block plus direction-specific load relation
	Optional self or same-substation	Neutral physical block plus the corresponding relation
`heterogeneous_line`	Busbar ↔ line node	One origin/extremity-and-direction relation; no line-state attributes
	Generator ↔ busbar	One direction-specific generator relation
	Load ↔ busbar	One direction-specific load relation
	Optional self or same-substation	One corresponding relation
Any augmented schema	Busbar → summary/virtual, with optional reverse	Zero edge vector except neutral `rho`= 1 when present; no dedicated relation one-hot in the ordinary GINE/GAT encoder

The complete relation names and activation rules are given in Tables 3.6 and 3.8. Those tables distinguish, for example, generator-to-busbar from busbar-to-generator and origin from extremity line attachments. These distinctions matter directly to both edge-aware models: GINE changes message content and GAT changes attention.

Appendix B

Graph Construction Details

This appendix collects the construction and sizing details of the three graph schemas of Section 3.2: how large each candidate graph is, how inactive candidates are neutralized, and how a local actor graph is cut out of the full grid. None of it changes the representations themselves.

B.1 Candidate graph sizes

Let S be the number of included substations, B the busbars per substation, L the included physical lines, and N_g, N_d the generators and loads. Write $d_g, d_d, d_\ell \in \{1, 2\}$ for the number of retained directions of the generator, load, and line-node attachments: 2 for `bidirectional` and 1 for either one-way mode. Table B.1 gives the size of the fixed candidate graph, before any mask is applied.

Table B.1: Size of the candidate graph of each schema. Enabling self edges adds $|\mathcal{V}|$ directed edges and enabling same-substation edges adds $SB(B-1)$, in every schema.

Schema	Candidate nodes	Candidate directed edges
`bus`	SB	$2B^2L$
`heterogeneous`	$SB + N_g + N_d$	$2B^2L + B(d_gN_g + d_dN_d)$
`heterogeneous_line`	$SB + N_g + N_d + L$	$2Bd_\ell L + B(d_gN_g + d_dN_d)$

Three consequences are worth naming. A busbar graph with both optional relations enabled therefore has $2B^2L + SB + SB(B-1)$ candidate directed edges. The disaggregated schema adds $N_g + N_d$ nodes and between $B(N_g + N_d)$ and $2B(N_g + N_d)$ asset candidates to the busbar graph, and buys individual asset states with them. The explicit-line schema replaces the $2B^2L$ busbar-to-busbar term by $2Bd_\ell L$: identical at $B = 2$ and cheaper for $B > 2$, at the cost of L extra nodes and one extra message-passing hop between adjacent busbars.

The hierarchical augmentations of Section 3.3 add on top of any of these: one substation-summary node per substation, with SB directed edges for `toward_summary` or $2SB$ for `bidirectional`; and one virtual node, with N_{bus} or $2N_{\text{bus}}$ edges, or S or $2S$ when summary nodes are also enabled.

B.2 Indexing and masking conventions

Grid2Op numbers busbars from one in its topology vector; those identifiers are converted to the zero-based index b used in Equation 3.6.

An inactive candidate edge keeps its row in the edge-feature tensor, so the tensor shape is fixed for an episode, but the row is zero-filled and its `edge_mask` entry is 0. Masked edges are removed before the encoder is called, so those zero rows never generate a message. This is what allows a topology change to alter connectivity without rebuilding the graph.

B.3 Local actor graphs

A local actor graph contains the controlled substations plus, optionally, their one-hop neighbors. Nodes of the controlled substations form the action domain, and the neighbors provide context only; the centralized state graph always contains the full grid. In the explicit-line schema a line node counts as controlled when either of its endpoints is controlled. The same controlled-domain and one-hop rules apply to all three schemas.

Appendix C

Experiment Configurations

This appendix provides the complete run configurations for the GINE encoder and heavy GINE ablations.

Table C.1: GINE rerun configuration summary. The reference run is `Heavy GINE Baseline`.

Run	Actor/critic encoder	GNN hidden/layers	Actor/critic heads	Concat flat	Edge pre-enc.	Entropy	Init action-0	Opt. critic	Difference from `Heavy GINE Baseline`
`Heavy GINE Baseline`	`gnn` / `gnn`	128 / 2	3×256 / 3×256	true	true	0.02	0.7	false	Baseline: heavy concat-flat GINE actor, GNN critic, and legacy critic updates
`Light GINE (Concat, MLP Critic)`	`gnn` / `mlp`	64 / 1	2×128 / 2×128	true	false	0.02	0.7	true	Light GINE actor, MLP critic, smaller heads, no edge pre-encoder, optimized critic updates
`Heavy GINE (No Bias)`	`gnn` / `gnn`	128 / 2	3×256 / 3×256	true	true	0.02	0.0	false	Bias ablation: same as baseline, but the initial do-nothing prior is 0.0
`No Bias, Opt Critic`	`gnn` / `gnn`	128 / 2	3×256 / 3×256	true	true	0.02	0.0	true	Bias 0.0 plus optimized critic updates; otherwise heavy concat-flat GINE with GNN critic
`Optimized Heavy GINE`	`gnn` / `gnn`	128 / 2	3×256 / 3×256	true	true	0.01	0.0	true	Optimized heavy GINE: bias 0.0, lower entropy 0.01, optimized critic updates
`Optimized Light GINE`	`gnn` / `mlp`	64 / 1	2×128 / 2×128	false	false	0.01	0.0	true	Light optimized GINE: MLP critic, no concat-flat, lower entropy 0.01, bias 0.0, optimized critic updates

Table C.2: Heavy-GINE parameter comparison from `configs/gine_s0_s1_s2`.

Run	Main change from baseline	GNN	GNN size	Critic	Actor/critic heads	Concat	Shared	Edge pre.	Node ID	Entropy	Init action-0	Opt. critic
`No Flat Concat`	Removes flat-observation concatenation	`gine`	2 × 128	`gnn`	3 × 256 / 3 × 256	no	yes	yes	yes	0.02–0.02	0.7	no
`Heavy GINE Baseline`	Reference: heavy concat-flat GINE with GNN critic	`gine`	2 × 128	`gnn`	3 × 256 / 3 × 256	yes	yes	yes	yes	0.02–0.02	0.7	no
`Opt. Critic Updates`	Enables optimized critic updates	`gine`	2 × 128	`gnn`	3 × 256 / 3 × 256	yes	yes	yes	yes	0.02–0.02	0.7	yes
`MLP Critic`	Replaces the GNN critic with an MLP critic	`gine`	2 × 128	`mlp`	3 × 256 / 3 × 256	yes	yes	yes	yes	0.02–0.02	0.7	no
`Unshared Actor`	Uses non-shared actor GNN weights	`gine`	2 × 128	`gnn`	3 × 256 / 3 × 256	yes	no	yes	yes	0.02–0.02	0.7	no
`Light GINE Encoder`	Uses a lighter GINE encoder	`gine`	1 × 64	`gnn`	3 × 256 / 3 × 256	yes	yes	yes	yes	0.02–0.02	0.7	no
`Zero Entropy Decay`	Decays entropy from 0.02 to 0.0	`gine`	2 × 128	`gnn`	3 × 256 / 3 × 256	yes	yes	yes	yes	0.02–0.00	0.7	no
`No Node-ID Embs`	Removes learned node-ID embeddings	`gine`	2 × 128	`gnn`	3 × 256 / 3 × 256	yes	yes	yes	no	0.02–0.02	0.7	no
`GAT Message Passing`	Replaces GINE with GAT message passing	`gat`	2 × 128	`gnn`	3 × 256 / 3 × 256	yes	yes	yes	yes	0.02–0.02	0.7	no
`Weighted GCN`	Replaces GINE with weighted GCN message passing	`gcn`	2 × 128	`gnn`	3 × 256 / 3 × 256	yes	yes	yes	yes	0.02–0.02	0.7	no
`Light GINE (No Concat, MLP Critic)`	Light no-concat GINE with MLP critic and optimized critic updates	`gine`	1 × 64	`mlp`	2 × 128 / 2 × 128	no	yes	no	yes	0.02–0.02	0.7	yes
`Light GINE (Concat, MLP Critic)`	Light concat-flat GINE with MLP critic and optimized critic updates	`gine`	1 × 64	`mlp`	2 × 128 / 2 × 128	yes	yes	no	yes	0.02–0.02	0.7	yes
`No Initial Bias`	Removes the initial do-nothing bias	`gine`	2 × 128	`gnn`	3 × 256 / 3 × 256	yes	yes	yes	yes	0.02–0.02	0.0	no

Appendix D

Full-Test Checkpoint Registry

Table D.1 reports the checkpoint step used for each full-test result table. The values are the `checkpoint_global_step` fields stored in the corresponding JSON files under `Topology_Task/outputs/full_test_eval`. The do-nothing policy has no learned checkpoint. For the behavioral cloning students, the stored step is the student checkpoint's optimizer-step count rather than a MAPPO environment step.

Table D.1: Checkpoint steps used for the full-test evaluations.

Run type	Seed 0 checkpoint step	Seed 1 checkpoint step	Seed 2 checkpoint step
Do-nothing policy	–	–	–
Baseline flat policy	13,600,000	13,040,000	13,760,000
Global rho heuristic, $\rho_{safe} = 0.90$	13,920,000	14,320,000	14,800,000
Local rho heuristic, $\rho_{safe} = 0.90$	14,160,000	14,400,000	14,160,000
Intervention gate, final-action MAP	12,000,000	12,720,000	13,760,000
Intervention gate, hierarchical greedy	7,120,000	14,720,000	13,760,000
Flat Sparse16, $\lambda_{fixed} = 0.000$	9,600,000	12,160,000	7,920,000
Flat Sparse16, $\lambda_{fixed} = 0.001$	10,640,000	11,520,000	11,600,000
Flat Sparse16, $\lambda_{fixed} = 0.003$	12,000,000	12,960,000	12,480,000
Flat Sparse16, $\lambda_{fixed} = 0.010$	8,960,000	5,760,000	12,560,000
Gated Sparse16, $\lambda_{fixed} = 0.000$	8,480,000	4,320,000	6,000,000
Gated Sparse16, $\lambda_{fixed} = 0.001$	5,760,000	7,600,000	3,280,000
Gated Sparse16, $\lambda_{fixed} = 0.003$	8,960,000	7,120,000	7,600,000
Gated Sparse16, $\lambda_{fixed} = 0.010$	12,000,000	7,840,000	7,920,000
Flat local-safe AIB, $d = 0.20$	13,120,000	9,360,000	12,160,000
Flat local-safe AIB, $d = 0.10$	12,880,000	13,200,000	12,720,000
Flat local-safe AIB, $d = 0.35$	12,800,000	5,360,000	11,520,000
Gated local-safe AIB, $d = 0.20$, hierarchical greedy	14,960,000	5,280,000	16,800,000
Flat non-idle AIB, $d = 0.20$	12,480,000	14,880,000	11,920,000
Teacher: MAPPO with local rho heuristic	14,160,000	14,400,000	14,160,000
Student BC, balanced non-idle 0.50, weight 5, aux 0.50	12,375	14,994	13,617
Student BC, balanced non-idle 0.20, weight 3, aux 0.25	12,375	14,994	13,617

Bibliography

[1] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" *International Conference on Learning Representations*, 2019.

[2] B. Donnot, "Grid2op: A testbed platform to model sequential decision making in power systems," *arXiv preprint arXiv:2011.07929*, 2020.

[3] C. Yu et al., "The surprising effectiveness of ppo in cooperative multi-agent games," *Advances in Neural Information Processing Systems*, 2022.

[4] J. Manczak et al., *Hierarchical reinforcement learning for power network topology control*, 2023. arXiv: 2311.02129. [Online]. Available: https://arxiv.org/abs/2311.02129

[5] E. van der Sar et al., *Hierarchical multi-agent reinforcement learning for power grid topology optimization*, 2023. arXiv: 2310.02605. [Online]. Available: https://arxiv.org/abs/2310.02605

[6] J. Achiam, D. Held, A. Tamar, and P. Abbeel, "Constrained policy optimization," *International Conference on Machine Learning*, 2017. [Online]. Available: https://arxiv.org/abs/1705.10528

[7] A. Stooke, J. Achiam, and P. Abbeel, "Responsive safety in reinforcement learning by PID lagrangian methods," *International Conference on Machine Learning*, 2020. [Online]. Available: https://arxiv.org/abs/2007.03964

[8] N. Carrara, E. Leurent, R. Laroche, T. Urvoy, and O.-A. Maillard, "Budgeted reinforcement learning in continuous state space," *Advances in Neural Information Processing Systems*, 2019. [Online]. Available: https://arxiv.org/abs/1903.01004

[9] L. Lautenbacher et al., *Multi-objective reinforcement learning for power grid topology control*, 2025. arXiv: 2502.00040. [Online]. Available: https://arxiv.org/abs/2502.00040

[10] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[11] A. Marot et al., "Learning to run a power network challenge for training topology controllers," *Electric Power Systems Research*, 2021.

[12] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," *International Conference on Learning Representations*, 2017.